

CAPITOLO 6

RAG e Agenti

Per risolvere un task, un modello ha bisogno sia delle istruzioni su come farlo, sia delle informazioni necessarie per farlo. Proprio come un essere umano ha più probabilità di dare una risposta sbagliata quando mancano informazioni, i modelli di IA hanno più probabilità di commettere errori e *hallucinate* quando manca loro il *context*. Per una data applicazione, le istruzioni del modello sono comuni a tutte le *query*, mentre il *context* è specifico per ogni *query*. Il capitolo precedente ha discusso come scrivere buone istruzioni per il modello. Questo capitolo si concentra su come costruire il *context* rilevante per ogni *query*.

I due modelli dominanti per la costruzione del *context* sono *RAG*, o *retrieval-augmented generation*, e gli *agents*. Il modello *RAG* permette al modello di recuperare informazioni rilevanti da fonti di dati esterne. Il modello *agentic* permette al modello di utilizzare *tools* come la ricerca web e le *API* di notizie per raccogliere informazioni.

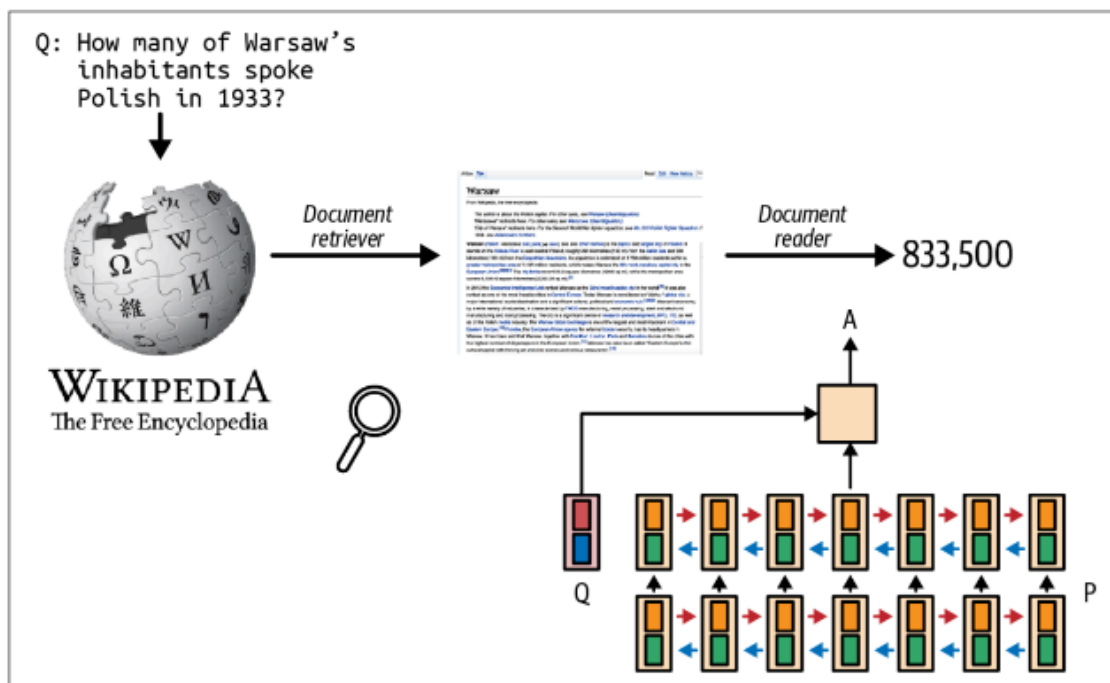
Mentre il modello *RAG* è utilizzato principalmente per costruire il *context*, il modello *agentic* può fare molto di più. Gli *tools* esterni possono aiutare i modelli a affrontare le loro carenze ed espandere le loro capacità. Più importante, danno ai modelli la capacità di interagire direttamente con il mondo, permettendo loro di automatizzare molti aspetti della nostre vite.

Sia i modelli *RAG* che quelli *agentic* sono entusiasmanti a causa delle capacità che portano a modelli già potenti. In un breve lasso di tempo, sono riusciti a catturare l'immaginazione collettiva, portando a *demo* e prodotti incredibili che convincono molte persone che sono il futuro. Questo capitolo entrerà nel dettaglio di ciascuno di questi modelli, come funzionano e cosa li rende così promettenti.

RAG

RAG è una tecnica che migliora la *generation* di un modello recuperando le informazioni rilevanti da fonti di *memory* esterne. Una fonte di *memory* esterna può essere un database interno, le sessioni di *chat* precedenti di un utente o *internet*.

Il modello *retrieve-then-generate* è stato introdotto per la prima volta in "Reading Wikipedia to Answer Open-Domain Questions" (Chen et al., 2017). In questo lavoro, il sistema recupera prima le cinque pagine di Wikipedia più rilevanti per una domanda, poi un modello¹ utilizza, o legge, le informazioni da queste pagine per generare una risposta, come visualizzato nella Figura 6-1.



[Figura 6-1: Il modello *retrieve-then-generate*. Il modello era indicato come il *document reader*.]

Il termine *retrieval-augmented generation* è stato coniato in "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks" (Lewis et al., 2020). Il documento proponeva RAG come soluzione per *knowledge-intensive tasks* dove tutta la conoscenza disponibile non può essere inserita direttamente nel modello. Con RAG, solo le informazioni più rilevanti per la *query*, come determinato dal *retriever*, vengono recuperate e inserite nel modello. Lewis et al. hanno scoperto che avere accesso a informazioni rilevanti può aiutare il modello a generare risposte più dettagliate riducendo al contempo le *hallucinations*.²

¹ Il modello utilizzato era un tipo di *recurrent neural network* nota come LSTM (*Long Short-Term Memory*). LSTM era l'architettura dominante del *deep learning* per l'elaborazione del *linguaggio naturale* (NLP) prima che l'architettura *transformer* prendesse il sopravvento nel 2018.

² Più o meno nello stesso periodo, un altro articolo, sempre di Facebook, "How Context Affects Language Models' Factual Predictions" (Petrone et al., arXiv, maggio 2020), ha mostrato che l'*augmenting* di un *pre-trained language model* con un *retrieval system* può migliorare drasticamente le prestazioni del modello su domande fattuali.

Ad esempio, data la *query* "La **fancy-printer-A300** della Acme può stampare 100pps?", il modello sarà in grado di rispondere meglio se gli vengono fornite le specifiche della **fancy-printer-A300**.³

Puoi pensare a RAG come a una tecnica per costruire un *context* specifico per ogni *query*, invece di usare lo stesso *context* per tutte le *query*. Questo aiuta a gestire i dati dell'utente, poiché ti permette di includere dati specifici di un utente solo nelle *query* relative a quell'utente.

La costruzione del *context* per i *foundation model* è equivalente all'*feature engineering* per i modelli *ML* classici. Servono allo stesso scopo: dare al modello le informazioni necessarie per elaborare un *input*.

Nei primi giorni dei *foundation model*, RAG è emerso come uno dei modelli più comuni. Il suo scopo principale era superare le limitazioni di *context* dei modelli. Molte persone pensano che un *context* sufficientemente lungo sarà la fine di RAG. Io non la penso così. Primo, non importa quanto sia lunga la *context length* di un modello, ci saranno applicazioni che richiederanno un *context* più lungo di quello. Dopotutto, la quantità di dati disponibili cresce solo nel tempo. Le persone generano e aggiungono nuovi dati ma raramente cancellano dati. La *context length* si sta espandendo rapidamente, ma non abbastanza velocemente per le esigenze di dati di applicazioni arbitrarie.⁴

Secondo, un modello che può elaborare un *context* lungo non necessariamente utilizza bene quel *context*, come discusso in "*Context Length ed Efficiency del Context*" a pagina 218. Più lungo è il *context*, più è probabile che il modello si concentri sulla parte sbagliata del *context*. Ogni *token* di *context* extra comporta un costo extra e ha il potenziale di aggiungere *latency* extra. RAG permette a un modello di utilizzare solo le informazioni più rilevanti per ogni *query*, riducendo il numero di *input token* mentre potenzialmente aumenta le prestazioni del modello.

Gli sforzi per espandere la *context length* stanno avvenendo in parallelo con gli sforzi per far sì che i modelli utilizzino il *context* in modo più efficace. Non sarei sorpreso se un fornitore di modelli incorporasse un meccanismo simile al *retrieval* o all'*attention* per aiutare un modello a selezionare le parti più salienti di un *context* da utilizzare.

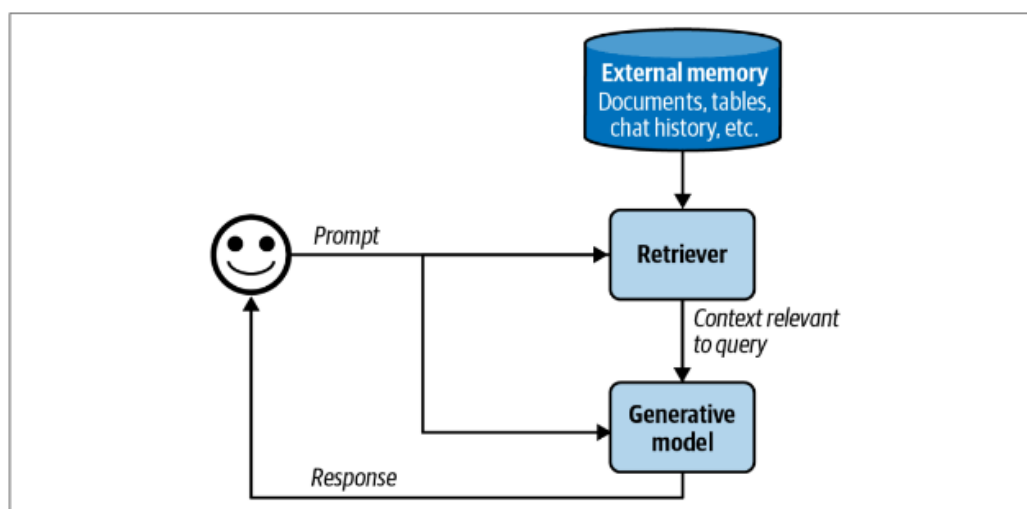
³ Grazie a Chetan Tekur per l'esempio.

⁴ La Legge di Parkinson è solitamente espressa come "Il lavoro si espande in modo da riempire il tempo disponibile per il suo completamento". Io ho una teoria simile: il *context* di un'applicazione si espande per riempire il limite di *context* supportato dal modello che utilizza.

Anthropic ha suggerito che per i modelli Claude, se "la tua *knowledge base* è più piccola di 200.000 *token* (circa 500 pagine di materiale), puoi semplicemente includere l'intera *knowledge base* nel *prompt* che dai al modello, senza bisogno di RAG o metodi simili" (Anthropic, 2024). Sarebbe fantastico se altri sviluppatori di modelli fornissero linee guida simili per RAG versus *long context* per i loro modelli.

Architettura RAG

Un sistema RAG ha due componenti: un *retriever* che recupera informazioni da fonti di *memory* esterne e un *generator* che genera una risposta basata sulle informazioni recuperate. La Figura 6-2 mostra un'architettura di alto livello di un sistema RAG.



[Figura 6-2: Un'architettura RAG di base.]

Nel documento RAG originale, Lewis et al. hanno addestrato insieme il *retriever* e il *generative model*. Nei sistemi RAG odierni, questi due componenti sono spesso addestrati separatamente, e molti team costruiscono i loro sistemi RAG utilizzando *retriever* e modelli predefiniti. Tuttavia, il *finetuning* dell'intero sistema RAG *end-to-end* può migliorare significativamente le sue prestazioni.

Il successo di un sistema RAG dipende dalla qualità del suo *retriever*. Un *retriever* ha due funzioni principali: *indexing* e *querying*. L'*indexing* implica l'elaborazione dei dati in modo che possano essere rapidamente recuperati in seguito. L'invio di una *query* per recuperare dati rilevanti per essa è chiamato *querying*. Come *indicizzare* i dati dipende da come si desidera recuperarli in seguito.

Ora che abbiamo coperto i componenti primari, consideriamo un esempio di come funziona un sistema RAG. Per semplicità, supponiamo che la *memory* esterna sia un *database* di documenti, come *memo* aziendali, contratti e appunti di riunioni. Un

documento può essere di 10 *token* o 1 milione di *token*. Recuperare ingenuamente interi documenti può far sì che il tuo *context* diventi arbitrariamente lungo. Per evitare ciò, puoi suddividere ogni documento in *chunk* più gestibili. Le strategie di *chunking* saranno discusse più avanti in questo capitolo. Per ora, supponiamo che tutti i documenti siano stati suddivisi in *chunk* utilizzabili. Per ogni *query*, il nostro obiettivo è recuperare i *chunk* di dati più rilevanti per questa *query*. Spesso è necessario un *post-processing* minore per unire i *chunk* di dati recuperati con il *prompt* dell'utente per generare il *prompt* finale. Questo *prompt* finale viene poi alimentato nel *generative model*.

In questo capitolo, uso il termine "documento" per riferirmi sia a "documento" che a "*chunk*", perché tecnicamente, un *chunk* di un documento è anche un documento. Lo faccio per mantenere le terminologie di questo libro coerenti con le terminologie classiche dell'*NLP* e dell'*information retrieval (IR)*.

Algoritmi di Retrieval

Il *retrieval* non è unico per *RAG*. L'*information retrieval* è un'idea centenaria.⁵ È l'ossatura dei motori di ricerca, dei sistemi di raccomandazione, dell'analisi dei log, ecc. Molti algoritmi di *retrieval* sviluppati per i sistemi di *retrieval* tradizionali possono essere utilizzati anche per *RAG*. Ad esempio, l'*information retrieval* è un'area di ricerca fertile con un grande settore di supporto che difficilmente può essere coperto sufficientemente in poche pagine. Di conseguenza, questa sezione coprirà solo le linee generali. Vedi il *repository GitHub* di questo libro per risorse più approfondite sull'*information retrieval*.

Il *retrieval* è tipicamente limitato a un *database* o sistema, mentre la ricerca implica il *retrieval* attraverso vari sistemi. Questo capitolo usa *retrieval* e ricerca in modo intercambiabile.

Fondamentalmente, il *retrieval* funziona classificando i documenti in base alla loro rilevanza per una data *query*. Gli algoritmi di *retrieval* differiscono in base al modo in cui vengono calcolati i punteggi di rilevanza. Inizierò con due meccanismi di *retrieval* comuni: *term-based retrieval* e *embedding-based retrieval*.

⁵ L'*information retrieval* è stato descritto già negli anni '20 nei brevetti di Emanuel Goldberg per una "macchina statistica" per cercare documenti memorizzati su pellicole. Vedi "The History of Information Retrieval Research" (Sanderson and Croft, Proceedings of the IEEE, 100: Special Centennial Issue, April 2012).

Retrieval Sparso versus Denso

In letteratura, potresti incontrare la divisione degli algoritmi di *retrieval* nelle seguenti categorie: *sparse* versus *dense*. Questo libro, tuttavia, ha optato per la categorizzazione *term-based* versus *embedding-based*.

I *retriever sparsi* rappresentano i dati usando *vettori sparsi*. Un *vettore sparso* è un vettore dove la maggior parte dei valori sono 0. Il *retrieval basato su termini* è considerato *sparse*, poiché ogni termine può essere rappresentato usando un vettore *one-hot* sparso, un vettore che è 0 ovunque tranne un valore di 1. La dimensione del vettore è la lunghezza del vocabolario. Il valore di 1 è nell'indice corrispondente all'indice del termine nel vocabolario.

Se abbiamo un dizionario semplice, {"cibo": 0, "banana": 1, "lumaca": 2}, allora i vettori *one-hot* di "cibo", "banana" e "lumaca" sono [1, 0, 0], [0, 1, 0] e [0, 0, 1], rispettivamente.

I *retriever densi* rappresentano i dati usando *vettori densi*. Un *vettore denso* è un vettore dove la maggior parte dei valori non sono 0. Il *retrieval basato su embedding* è tipicamente considerato *dense*, poiché gli *embedding* sono generalmente vettori densi. Tuttavia, esistono anche *embedding sparsi*. Ad esempio, *SPLADE* (*Sparse Lexical and Expansion*) è un algoritmo di *retrieval* che funziona utilizzando *embedding sparsi* (Formal et al., 2021). Sfrutta gli *embedding* generati da *BERT* ma utilizza la *regularization* per spingere la maggior parte dei valori degli *embedding* a 0. La *sparsità* rende le operazioni di *embedding* più efficienti.

La divisione *sparse* versus *dense* fa sì che *SPLADE* venga raggruppato con gli algoritmi *term-based*, anche se le operazioni, i punti di forza e le debolezze di *SPLADE* sono molto più simili a quelli del *retrieval di embedding densi* che a quelli del *retrieval basato su termini*. La divisione *term-based* versus *embedding-based* evita questa errata categorizzazione.

Retrieval basato su termini

Data una *query*, il modo più semplice per trovare documenti rilevanti è con le parole chiave. Alcune persone chiamano questo approccio *lexical retrieval*. Ad esempio, data la *query* "AI engineering", il modello recupererà tutti i documenti che contengono "AI engineering". Tuttavia, questo approccio ha due problemi:

- Molti documenti potrebbero contenere il termine dato e il tuo modello potrebbe non avere spazio di *context* sufficiente per includerli tutti come *context*. Un'euristica è includere i documenti che contengono il termine il maggior numero di volte. L'ipotesi è che più un termine appare in un documento, più questo documento è rilevante per questo termine. Il numero di volte che un termine appare in un documento è chiamato *term frequency (TF)*.
- Un *prompt* può essere lungo e contenere molti termini. Alcuni sono più importanti di altri. Ad esempio, il *prompt* "Ricette facili da seguire per cucinare cibo vietnamita a casa" contiene nove termini: *easy-to-follow*, *recipes*, *for*, *Vietnamese*, *food*,

to, cook, at, home. Vuoi concentrarti su termini più informativi come *vietnamese* e *recipes*, non su *for* e *at*. Hai bisogno di un modo per identificare i termini importanti.

L'intuizione è che più documenti contengono un termine, meno informativo è questo termine. "For" e "at" probabilmente appaiono nella maggior parte dei documenti, quindi sono meno informativi. Quindi l'importanza di un termine è inversamente proporzionale al numero di documenti in cui appare. Questa metrica è chiamata *inverse document frequency (IDF)*. Per calcolare l'*IDF* per un termine, conta tutti i documenti che contengono questo termine, poi dividi il numero totale di documenti per questo conteggio. Se ci sono 10 documenti e 5 di loro contengono un dato termine, allora l'*IDF* di questo termine è $10/5=2$. Più alto è l'*IDF* di un termine, più importante è. *TF-IDF* è un algoritmo che combina queste due metriche: *term frequency (TF)* e *inverse document frequency (IDF)*. Matematicamente, il punteggio *TF-IDF* del documento D per la query Q è calcolato come segue:

- Sia $t_1, t_2, \dots, t_q$ i termini nella query Q.
- Dato un termine t , la *term frequency* di questo termine nel documento D è $f(t, D)$.
- Sia N il numero totale di documenti e $C(t)$ il numero di documenti che contengono t. Il valore *IDF* del termine t può essere scritto come $IDF(t) = \log \frac{N}{C(t)}$.
- Naif, il punteggio *TF-IDF* di un documento D rispetto a Q è definito come $Score(D, Q) = \sum_{i=1}^q IDF(t_i) \times f(t_i, D)$.

Due soluzioni comuni di *retrieval* basate su termini sono *Elasticsearch* e *BM25*. *Elasticsearch* (Shay Banon, 2010), costruito su *Lucene*, utilizza una struttura dati chiamata *inverted index*. È un dizionario che mappa dai termini ai documenti che li contengono. Questo dizionario permette un recupero rapido dei documenti dato un termine. L'*index* potrebbe anche memorizzare informazioni aggiuntive come la *term frequency* e il *document count* (quanti documenti contengono questo termine), che sono utili per calcolare i punteggi *TF-IDF*. La Tabella 6-1 illustra un *inverted index*.

Termine	Conteggio doc.	(Indice doc., <i>term freq.</i>) per tutti i doc. contenenti il termine
banana	2	(10, 3), (5, 2)
machine	4	(1, 5), (10, 1), (38, 9), (42, 5)
learning	3	(1, 5), (38, 7), (42, 5)
...	...	...

Okapi BM25, la 25esima generazione dell'algoritmo *Best Matching*, è stato sviluppato da Robertson et al. negli anni '80. Il suo *scorer* è una modifica di *TF-IDF*. Rispetto al *TF-IDF* naif, *BM25* normalizza i punteggi di *term frequency* per la lunghezza del documento. Documenti più lunghi

hanno più probabilità di contenere un dato termine e hanno valori di *term frequency* più alti.⁶

BM25 e le sue varianti (**BM25+*, *BM25F**) sono ancora ampiamente utilizzati nel settore e servono come *baseline* formidabili per confrontarsi con algoritmi di *retrieval* più moderni e sofisticati, come il *retrieval basato su embedding*, discusso in seguito.⁷

Un processo che ho sorvolato è la *tokenization*, il processo di scomporre una *query* in singoli termini. Il metodo più semplice è dividere la *query* in parole, trattando ogni parola come un termine separato. Tuttavia, questo può portare a termini multi-parola essere scomposti in singole parole, perdendo il loro significato originale. Ad esempio, "hot dog" sarebbe diviso in "hot" e "dog". Quando ciò accade, nessuno dei due conserva il significato del termine originale. Un modo per mitigare questo problema è trattare gli *n-grammi* più comuni come termini. Se il *bigramma* "hot dog" è comune, sarà trattato come un termine.

Inoltre, potresti voler convertire tutti i caratteri in minuscolo, rimuovere la punteggiatura ed eliminare le *stop word* (come "the", "and", "is", ecc.). Le soluzioni di *retrieval* basate su termini spesso gestiscono queste cose automaticamente. I pacchetti *NLP* classici, come *NLTK* (*Natural Language Toolkit*), *spaCy* e *Stanford's CoreNLP*, offrono anche funzionalità di *tokenization*.

Il Capitolo 4 discute la misurazione della *similarità lessicale* tra due testi basata sulla loro sovrapposizione di *n-grammi*. Possiamo recuperare documenti in base all'estensione della loro sovrapposizione di *n-grammi* con la *query*? Sì, possiamo. Questo approccio funziona meglio quando la *query* e i documenti hanno lunghezze simili. Se i documenti sono molto più lunghi della *query*, la probabilità che contengano gli *n-grammi* della *query* aumenta, portando a molti documenti con punteggi di sovrapposizione similmente alti. Questo rende difficile distinguere i documenti veramente rilevanti da quelli meno rilevanti.

Retrieval basato su embedding

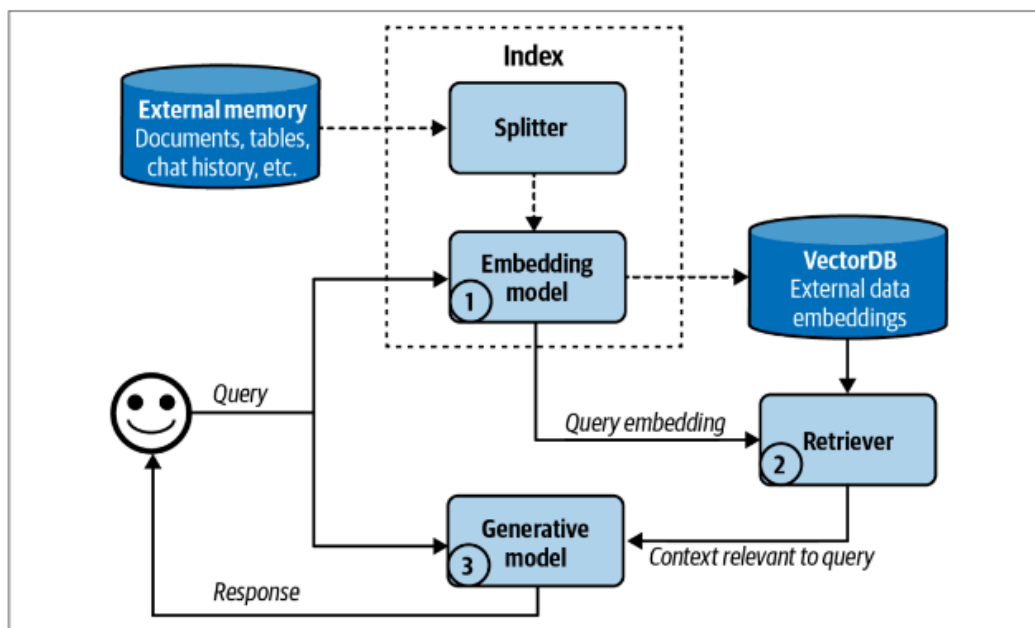
Il *retrieval basato su termini* calcola la rilevanza a livello lessicale piuttosto che a livello semantico. Come menzionato nel Capitolo 3, l'aspetto di un testo non necessariamente cattura il suo significato. Ciò può comportare la restituzione di documenti irrilevanti per la tua intenzione. Ad esempio, interrogare "*transformer architecture*" potrebbe restituire documenti sul dispositivo elettrico o sul film *Transformers*. D'altra parte, i *retriever* basati su *embedding* mirano a classificare i documenti in base a quanto strettamente i loro significati si allineano con la *query*. Questo approccio è anche noto come *semantic retrieval*.

⁶ Per coloro che sono interessati a saperne di più su *BM25*, raccomando questo articolo degli autori di *BM25*: "The Probabilistic Relevance Framework: *BM25* and Beyond" (Robertson and Zaragoza, *Foundations and Trends in Information Retrieval* 3 No. 4, 2009)

⁷ Aravind Srinivas, il CEO di Perplexity, ha twittato che "Fare un miglioramento genuino rispetto a *BM25* o alla *full-text search* è difficile".

Con il *retrieval basato su embedding*, l'*indexing* ha una funzione extra: convertire i *chunk* di dati originali in *embedding*. Il *database* in cui sono memorizzati gli *embedding* generati è chiamato *vector database*. Il *querying* consiste quindi in due passi, come mostrato nella Figura 6-3:

1. Modello di embedding: converti la *query* in un *embedding* usando lo stesso modello di *embedding* utilizzato durante l'*indexing*.
2. Retriever: recupera k *chunk* di dati i cui *embedding* sono più vicini all'*embedding* della *query*, come determinato dal *retriever*. Il numero di *chunk* di dati da recuperare, k , dipende dal caso d'uso, dal modello generativo e dalla *query*.



[Figura 6-3: Una vista di alto livello di come funziona un *retriever* basato su *embedding*, o semantico.]

Il flusso di lavoro del *retrieval basato su embedding* mostrato qui è semplificato. I sistemi di *retrieval* semantico del mondo reale potrebbero contenere altri componenti, come un *reranker* per ri-classificare tutti i candidati recuperati e *cache* per ridurre la *latency*.⁸

Con il *retrieval basato su embedding*, incontriamo di nuovo gli *embedding*, che sono discussi nel Capitolo 3. Come promemoria, un *embedding* è tipicamente un vettore che mira a preservare le proprietà importanti dei dati originali. Un *retriever* basato su *embedding* non funziona se il modello di *embedding* è cattivo.

⁸ Un flusso di lavoro di *retrieval RAG* condivide molti passi simili con il tradizionale sistema di raccomandazione.

Il *retrieval basato su embedding* introduce anche un nuovo componente: i *vector database*. Un *vector database* memorizza vettori. Tuttavia, la memorizzazione è la parte facile di un *vector database*. La parte difficile è la *vector search*. Data un'*embedding* di *query*, un *vector database* è responsabile di trovare i vettori nel *database* vicini alla *query* e restituirli. I vettori devono essere *indicizzati* e memorizzati in un modo che renda la *vector search* veloce ed efficiente. Come molti altri meccanismi da cui dipendono le applicazioni di *AI generativa*, la *vector search* non è unica per l'*AI generativa*. La *vector search* è comune in qualsiasi applicazione che utilizza *embedding*: ricerca, raccomandazione, organizzazione dei dati, *information retrieval*, *clustering*, rilevamento di frodi e altro.

La *vector search* è tipicamente inquadrata come un problema di *nearest-neighbor search*. Ad esempio, data una *query*, trova i k vettori più vicini. La soluzione *naive* è *k-nearest neighbors* (*k-NN*), che funziona come segue:

1. Calcola i punteggi di *similarity* tra l'*embedding* della *query* e tutti i vettori nel *database*, utilizzando metriche come la *cosine similarity*.
2. Classifica tutti i vettori in base ai loro punteggi di *similarity*.
3. Restituisce i k vettori con i punteggi di *similarity* più alti.

Questa soluzione *naive* garantisce che i risultati siano precisi, ma è computazionalmente pesante e lenta. Dovrebbe essere utilizzata solo per *dataset* piccoli.

Per *dataset* grandi, la *vector search* viene tipicamente eseguita utilizzando un algoritmo di *approximate nearest neighbor* (*ANN*). A causa dell'importanza della *vector search*, molti algoritmi e librerie sono stati sviluppati per essa. Alcune librerie di *vector search* popolari sono *FAISS* (*Facebook AI Similarity Search*) (Johnson et al., 2017), *ScaNN* di Google (*Scalable Nearest Neighbors*) (Sun et al., 2020), *Annoy* di Spotify (Bernhardsson, 2013) e *Hnswlib* (*Hierarchical Navigable Small World*) (Malkov and Yashunin, 2016).

La maggior parte degli sviluppatori di applicazioni non implementerà la *vector search* da sé, quindi darò solo una rapida panoramica dei diversi approcci. Questa panoramica potrebbe essere utile mentre valuti le soluzioni. In generale, i *vector database* organizzano i vettori in *bucket*, alberi o grafi. Gli algoritmi di *vector search* differiscono in base alle *euristiche* che usano per aumentare la probabilità che vettori simili siano vicini tra loro. I vettori possono anche essere *quantizzati* (precisione ridotta) o resi *sparsi*. L'idea è che i vettori *quantizzati* e *sparsi* sono meno intensivi dal punto di vista computazionale. Per coloro che vogliono saperne di più sulla *vector search*, Zilliz ha un'eccellente serie su di essa. Ecco alcuni algoritmi significativi di *vector search*:

LSH (locality-sensitive hashing) (Indyk and Motwani, 1999)

Questo è un algoritmo potente e versatile che funziona con più dei soli vettori. Ciò comporta l'*hashing* di vettori simili negli stessi *bucket* per accelerare la ricerca di *similarity*, scambiando un po' di accuratezza per efficienza. È implementato in *FAISS* e *Annoy*.

HNSW (Hierarchical Navigable Small World) (Malkov and Yashunin, 2016)

HNSW costruisce un grafo a più livelli dove i nodi rappresentano vettori e gli archi connettono vettori simili, permettendo ricerche del *nearest neighbor* attraversando gli archi del grafo. La sua implementazione degli autori è *open source*, ed è anche implementata in *FAISS* e *Milvus*.

Product Quantization (Jégou et al., 2011)

Funziona riducendo ogni vettore in una rappresentazione molto più semplice e a dimensione inferiore scomponendo ogni vettore in più *subvectors*. Le distanze sono poi calcolate usando le rappresentazioni a dimensione inferiore, che sono molto più veloci da usare. La *product quantization* è un componente chiave di *FAISS* ed è supportata da quasi tutte le librerie di *vector search* popolari.

IVF (inverted file index) (Sivic and Zisserman, 2003)

IVF utilizza il *clustering K-means* per organizzare vettori simili nello stesso *cluster*. A seconda del numero di vettori nel *database*, è tipico impostare il numero di *cluster* in modo che, in media, ci siano da 100 a 10.000 vettori in ogni *cluster*. Durante il *querying*, *IVF* trova i *centroidi* del *cluster* più vicini all'*embedding* della *query*, e i vettori in questi *cluster* diventano *neighbor* candidati. Insieme alla *product quantization*, *IVF* forma l'ossatura di *FAISS*.

Annoy (Approximate Nearest Neighbors Oh Yeah) (Bernhardsson, 2013)

Annoy è un approccio basato su alberi. Costruisce più alberi binari, dove ogni albero divide i vettori in *cluster* usando criteri casuali, come disegnare casualmente una linea e dividere i vettori in due rami usando questa linea. Durante una ricerca, attraversa questi alberi per raccogliere *neighbor* candidati. Spotify ha *open-sourced* la sua implementazione.

Ci sono altri algoritmi, come *SPTAG* di Microsoft (*Space Partition Tree And Graph*), e *FLANN* (*Fast Library for Approximate Nearest Neighbors*).

Anche se i *vector database* sono emersi come una loro categoria con l'ascesa di *RAG*, qualsiasi *database* che può memorizzare vettori può essere chiamato *vector database*. Molti *database* tradizionali hanno esteso o estenderanno il supporto per la memorizzazione vettoriale e la *vector search*.

Confronto degli algoritmi di retrieval

A causa della lunga storia del *retrieval*, le sue molte soluzioni mature rendono sia il *retrieval basato su termini* che quello *basato su embedding* relativamente facili da iniziare. Ogni approccio ha i suoi pro e contro.

Il *retrieval basato su termini* è generalmente molto più veloce del *retrieval basato su embedding* sia durante l'*indexing* che la *query*. L'estrazione dei termini è più veloce della generazione di *embedding* e il mapping da un termine ai documenti che lo contengono può essere meno costoso dal punto di vista computazionale di una ricerca del *nearest neighbor*.

Il *retrieval basato su termini* funziona anche bene *out of the box*. Soluzioni come *Elasticsearch* e *BM25* hanno alimentato con successo molte applicazioni di ricerca e *retrieval*. Tuttavia, la sua semplicità significa anche che ha meno componenti che puoi modificare per migliorare le sue prestazioni.

Il *retrieval basato su embedding*, d'altra parte, può essere significativamente migliorato nel tempo per superare il *retrieval basato su termini*. Puoi fare il *finetuning* del modello di *embedding* e del *retriever*, sia separatamente, insieme o in congiunzione con il modello generativo. Tuttavia, convertire i dati in *embedding* può oscurare parole chiave, come codici di errore specifici, ad esempio *EADDRNOTAVAIL (99)*, o nomi di prodotti, rendendoli più difficili da cercare in seguito. Questa limitazione può essere affrontata combinando il *retrieval basato su embedding* con il *retrieval basato su termini*, come discusso più avanti in questo capitolo.

La qualità di un *retriever* può essere valutata in base alla qualità dei dati che recupera. Due metriche spesso utilizzate dai *framework* di valutazione *RAG* sono la *context precision* e la *context recall*, o *precision* e *recall* per brevità (la *context precision* è anche chiamata *context relevance*):

Context precision

Su tutti i documenti recuperati, quale percentuale è rilevante per la *query*?

Context recall

Su tutti i documenti che sono rilevanti per la *query*, quale percentuale viene recuperata?

Per calcolare queste metriche, si cura un *evaluation set* con un elenco di *query* di test e un insieme di documenti. Per ogni *query* di test, si annota ogni documento di test come rilevante o non rilevante. L'annotazione può essere fatta da umani o da *AI judges*. Si calcolano poi il punteggio di *precision* e *recall* del *retriever* su questo *evaluation set*.

In produzione, alcuni *framework RAG* supportano solo la *context precision*, non la *context recall*. Per calcolare la *context recall* per una data *query*, è necessario annotare la rilevanza di tutti i documenti nel *database* per quella *query*. La *context precision* è più semplice da calcolare. Hai solo bisogno di confrontare i documenti recuperati con la *query*, cosa che può essere fatta da un *AI judge*.

Se ti interessa la classificazione dei documenti recuperati, ad esempio, i documenti più rilevanti dovrebbero essere classificati per primi, puoi utilizzare metriche come *NDCG* (*normalized discounted cumulative gain*), *MAP* (*Mean Average Precision*) e *MRR* (*Mean Reciprocal Rank*). Per il *retrieval* semantico, devi anche valutare la qualità dei tuoi *embedding*. Come discusso nel Capitolo 3, gli *embedding* possono essere valutati indipendentemente—sono considerati buoni se documenti più simili hanno *embedding* più vicini. Gli *embedding* possono anche essere valutati in base a quanto bene funzionano per *task* specifici. Il *benchmark MTEB* (Muennighoff et al., 2023) valuta gli *embedding* per un'ampia gamma di *task* inclusi *retrieval*, classificazione e *clustering*.

La qualità di un *retriever* dovrebbe essere valutata anche nel contesto dell'intero sistema RAG. In definitiva, un *retriever* è buono se aiuta il sistema a generare risposte di alta qualità. La valutazione degli *output* dei modelli generativi è discussa nei Capitoli 3 e 4. Se la promessa di prestazioni di un sistema di *retrieval* semantico valga la pena di essere perseguita dipende da quanto si dà priorità a costo e *latency*, in particolare durante la fase di *querying*. Poiché gran parte della *latency* RAG proviene dalla generazione dell'*output*, specialmente per *output* lunghi, la *latency* aggiunta dalla generazione dell'*embedding* della *query* e dalla *vector search* potrebbe essere minima rispetto alla *latency* RAG totale. Anche così, la *latency* aggiunta può ancora influenzare l'esperienza utente.

Un'altra preoccupazione è il costo. Generare *embedding* costa denaro. Questo è soprattutto un problema se i tuoi dati cambiano frequentemente e richiedono una rigenerazione frequente degli *embedding*. Immagina di dover generare *embedding* per 100 milioni di documenti ogni giorno! A seconda del *vector database* che usi, l'archiviazione vettoriale e le *query* di *vector search* possono essere costose. Non è raro vedere che la spesa per il *vector database* di un'azienda sia un quinto o addirittura la metà della loro spesa per le *API* del modello.

La Tabella 6-2 mostra un confronto fianco a fianco del *retrieval basato su termini* e del *retrieval basato su embedding*.

	<i>Term-based retrieval</i>	<i>Embedding-based retrieval</i>
Velocità query	Molto più veloce del <i>retrieval basato su embedding</i>	La generazione dell' <i>embedding</i> della <i>query</i> e la <i>vector search</i> possono essere lente
Prestazioni	Prestazioni tipicamente forti <i>out of the box</i> , ma difficili da migliorare	Può superare il <i>retrieval basato su termini</i> con il <i>finetuning</i>
	Può recuperare documenti sbagliati a causa dell'ambiguità dei termini	Permette l'uso di <i>query</i> più naturali, poiché si concentra sulla semantica invece che sui termini
Costo	Molto più economico del <i>retrieval basato su embedding</i>	<i>Embedding</i> , archiviazione vettoriale e soluzioni di <i>vector search</i> possono essere costose

Con i sistemi di *retrieval*, puoi fare certi compromessi tra *indexing* e *querying*. Più dettagliato è l'*index*, più accurato sarà il processo di *retrieval*, ma il processo di *indexing* sarà più lento e consumerà più memoria. Immagina di costruire un *index* di potenziali clienti. Aggiungere più dettagli (ad es., nome, azienda, email, telefono, interessi) rende più facile trovare persone rilevanti ma richiede più tempo per la costruzione e richiede più archiviazione.

In generale, un *index* dettagliato come *HNSW* fornisce alta accuratezza e tempi di *query* veloci ma richiede tempo e memoria significativi per la costruzione. Al contrario, un *index* più semplice come *LSH* è più veloce e meno intensivo di memoria da creare, ma risulta in *query* più lente e meno accurate.

Il sito web *ANN-Benchmarks* confronta diversi algoritmi *ANN* su più *dataset* utilizzando quattro metriche principali, tenendo conto dei compromessi tra *indexing* e *querying*. Queste includono:

Recall

La frazione dei *nearest neighbor* trovati dall'algoritmo.

Query per second (QPS)

Il numero di *query* che l'algoritmo può gestire al secondo. Questo è cruciale per applicazioni ad alto traffico.

Build time

Il tempo richiesto per costruire l'*index*. Questa metrica è particolarmente importante se è necessario aggiornare frequentemente l'*index* (ad es., perché i tuoi dati cambiano).

Index size

La dimensione dell'*index* creato dall'algoritmo, che è cruciale per valutarne la scalabilità e i requisiti di archiviazione.

Inoltre, *BEIR (Benchmarking IR)* (Thakur et al., 2021) è un'*evaluation harness* per il *retrieval*. Supporta sistemi di *retrieval* attraverso 14 *benchmark* di *retrieval* comuni.

Per riassumere, la qualità di un sistema *RAG* dovrebbe essere valutata sia componente per componente che *end-to-end*. Per fare questo, dovresti fare le seguenti cose:

1. Valutare la qualità del *retrieval*.
2. Valutare gli *output RAG* finali.
3. Valutare gli *embedding* (per il *retrieval basato su embedding*).

Combinazione di algoritmi di retrieval

Dati i distinti vantaggi di diversi algoritmi di *retrieval*, un sistema di *retrieval* di produzione tipicamente combina diversi approcci. La combinazione del *retrieval basato su termini* e del *retrieval basato su embedding* è chiamata *hybrid search*.

Diversi algoritmi possono essere utilizzati in sequenza. Primo, un *retriever* economico e meno preciso, come un sistema *basato su termini*, recupera i candidati. Poi, un meccanismo più preciso ma più costoso, come *k-nearest neighbors*, trova il migliore di questi candidati. Questo secondo passo è anche chiamato *reranking*.

Ad esempio, dato il termine "*transformer*", puoi recuperare tutti i documenti che contengono la parola *transformer*, indipendentemente dal fatto che riguardino il dispositivo elettrico, l'architettura neurale o il film. Poi usi la *vector search* per trovare tra questi documenti quelli che sono effettivamente correlati alla tua *query* su *transformer*. Come altro esempio, considera la *query* "Chi è responsabile del maggior numero di vendite a X?" Primo, potresti recuperare tutti i documenti associati a X usando la parola chiave X. Poi, usi la *vector search* per recuperare il *context* associato a "Chi è responsabile del maggior numero di vendite?"

Diversi algoritmi possono anche essere utilizzati in parallelo come un *ensemble*. Ricorda che un *retriever* funziona classificando i documenti in base ai loro punteggi di rilevanza per la *query*. Puoi usare più *retriever* per recuperare candidati contemporaneamente, poi combinare queste diverse classificazioni insieme per generare una classificazione finale.

Un algoritmo per combinare diverse classificazioni è chiamato ***reciprocal rank fusion (RRF)*** (Cormack et al., 2009). Assegna a ogni documento un punteggio in base alla sua classificazione da parte di un *retriever*. Intuitivamente, se è classificato primo, il suo punteggio è $1/1 = 1$. Se è classificato secondo, il suo punteggio è $1/2 = 0.5$. Più alto è classificato, più alto è il suo punteggio.

Il punteggio finale di un documento è la somma dei suoi punteggi rispetto a tutti i *retriever*. Se un documento è classificato primo da un *retriever* e secondo da un altro *retriever*, il suo punteggio è $1 + 0.5 = 1.5$. Questo esempio è una sovrasemplificazione di *RRF*, ma mostra le basi. La formula effettiva per un documento *D* è più complicata, come segue:

$$\text{Punteggio}(D) = \sum_{i=1}^n \frac{1}{k + r_i(D)}$$

- *n* è il numero di liste classificate; ogni lista di classifica è prodotta da un *retriever*.
- $r_i(D)$ è il *rank* del documento dal *retriever* *i*
- *k* è una costante per evitare la divisione per zero e per controllare l'influenza dei documenti classificati più in basso. Un valore tipico per *k* è 60.

Ottimizzazione del Retrieval

A seconda del *task*, certe tattiche possono aumentare la possibilità che vengano recuperati documenti rilevanti. Quattro tattiche discusse qui sono la strategia di *chunking*, il *reranking*, la *query rewriting* e il *contextual retrieval*.

Strategia di chunking

Come i tuoi dati dovrebbero essere *indicizzati* dipende da come intendi recuperarli in seguito. L'ultima sezione ha coperto diversi algoritmi di *retrieval* e le loro rispettive strategie di *indexing*. Lì, la discussione si basava sull'ipotesi che i documenti fossero già stati suddivisi in *chunk* gestibili. In questa sezione, coprirò diverse strategie di *chunking*. Questa è una considerazione importante perché la strategia di *chunking* che usi può influenzare significativamente le prestazioni del tuo sistema di *retrieval*.

La strategia più semplice è suddividere i documenti in *chunk* di uguale lunghezza basati su una certa unità. Unità comuni sono caratteri, parole, frasi e paragrafi. Ad esempio, puoi dividere ogni documento in *chunk* di 2.048 caratteri o 512 parole. Puoi anche dividere ogni documento in modo che ogni *chunk* possa contenere un numero fisso di frasi (come 20 frasi) o paragrafi (come ogni paragrafo è il suo *chunk*).

Puoi anche dividere i documenti in modo ricorsivo usando unità progressivamente più piccole finché ogni *chunk* rientra nella tua dimensione massima del *chunk*. Ad esempio, puoi iniziare dividendo un documento in sezioni. Se una sezione è troppo lunga, dividila in paragrafi. Se un paragrafo è ancora troppo lungo, dividilo in frasi. Questo riduce la possibilità che testi correlati vengano interrotti arbitrariamente.

Documenti specifici potrebbero anche supportare strategie di *chunking* creative. Ad esempio, ci sono *splitter* sviluppati specialmente per diversi linguaggi di programmazione. I documenti di domande e risposte possono essere divisi per coppia domanda o risposta, dove ogni coppia costituisce un *chunk*. I testi cinesi potrebbero dover essere divisi in modo diverso dai testi inglesi.

Quando un documento è diviso in *chunk* senza sovrapposizione, i *chunk* potrebbero essere troncati nel mezzo di un *context* importante, portando alla perdita di informazioni critiche. Considera il testo "Ho lasciato un biglietto a mia moglie". Se è diviso in "Ho lasciato mia moglie" e "un biglietto", nessuno di questi due *chunk* trasmette le informazioni chiave del testo originale. La sovrapposizione assicura che le importanti informazioni di confine siano incluse in almeno un *chunk*. Se imposti la dimensione del *chunk* a 2.048 caratteri, puoi forse impostare la dimensione di sovrapposizione a 20 caratteri.

La dimensione del *chunk* non dovrebbe superare la *context length* massima del modello generativo. Per l'approccio *basato su embedding*, la dimensione del *chunk* non dovrebbe superare neanche il limite di *context* del modello di *embedding*.

Puoi anche suddividere i documenti usando i *token*, determinati dal *tokenizer* del modello generativo, come unità. Diciamo che vuoi usare Llama 3 come tuo modello generativo. Allora prima *tokenizzi* i documenti usando il *tokenizer* di Llama 3. Puoi poi dividere i documenti in *chunk* usando i *token* come confini. Il *chunking* per *token* rende più facile lavorare con i modelli a valle. Tuttavia, lo svantaggio di questo approccio è che se passi a un altro modello generativo con un *tokenizer* diverso, dovresti *reindicizzare* i tuoi dati.

Indipendentemente dalla strategia che scegli, le dimensioni del *chunk* contano. Una dimensione del *chunk* più piccola permette informazioni più diversificate. *Chunk* più piccoli significano che puoi inserire più *chunk* nel *context* del modello. Se dimezzi la dimensione del *chunk*, puoi inserire il doppio dei *chunk*. Più *chunk* possono fornire a un modello una gamma più ampia di informazioni, che può permettere al modello di produrre una risposta migliore.

Dimensioni del *chunk* più piccole, tuttavia, possono causare la perdita di informazioni importanti. Immagina un documento che contiene informazioni importanti sull'argomento X in tutto il documento, ma X è menzionato solo nella prima metà. Se dividi questo documento in due *chunk*, la seconda metà del documento potrebbe non essere recuperata e il modello non sarà in grado di utilizzare le sue informazioni.

Dimensioni del *chunk* più piccole possono anche aumentare il sovraccarico computazionale. Questo è soprattutto un problema per il *retrieval basato su embedding*. Dimezzare la dimensione del *chunk* significa che hai il doppio dei *chunk* da indicizzare e il doppio dei vettori di *embedding* da generare e memorizzare. Il tuo spazio di *vector search* sarà il doppio, il che può ridurre la velocità della *query*.

Non esiste una dimensione del *chunk* o una dimensione di sovrapposizione universalmente migliore. Devi sperimentare per trovare ciò che funziona meglio per te.

Reranking

Le classificazioni iniziali dei documenti generate dal *retriever* possono essere ulteriormente ri-classificate per essere più accurate. Il *reranking* è particolarmente utile quando è necessario ridurre il numero di documenti recuperati, sia per adattarli al *context* del modello sia per ridurre il numero di *input token*.

Un modello comune per il *reranking* è discusso in "Combinazione di algoritmi di *retrieval*" a pagina 266. Un *retriever* economico ma meno preciso recupera i candidati, poi un meccanismo più preciso ma più costoso ri-classifica questi candidati.

I documenti possono anche essere ri-classificati in base al tempo, dando un peso maggiore ai dati più recenti. Questo è utile per applicazioni *time-sensitive* come l'aggregazione di notizie, la *chat* con le tue email (ad es., un *chatbot* che può rispondere a domande sulle tue email) o l'analisi del mercato azionario.

Il *context reranking* differisce dal *reranking* di ricerca tradizionale in quanto la posizione esatta degli elementi è meno critica. Nella ricerca, il *rank* (ad es., primo o quinto) è cruciale. Nel *context reranking*, l'ordine dei documenti conta ancora perché influisce su quanto bene un modello può elaborarli. I modelli potrebbero comprendere meglio i documenti all'inizio e alla fine del *context*, come discusso in "*Context Length ed Efficiency del Context*" a pagina 218. Tuttavia, fintanto che un documento è incluso, l'impatto del suo ordine è meno significativo rispetto alla classifica di ricerca.

Riscrittura della query

Query rewriting è anche nota come *query reformulation*, *query normalization* e talvolta *query expansion*. Considera la seguente conversazione:

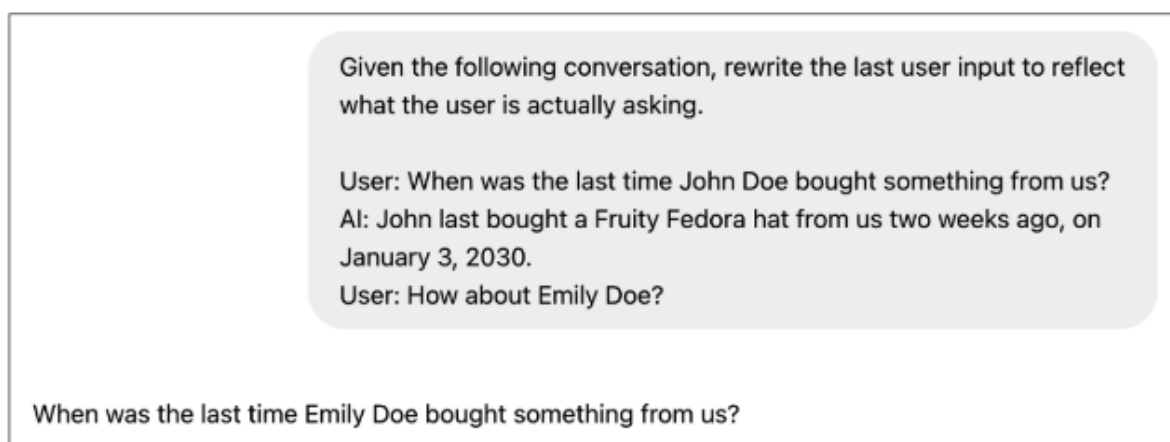
Utente: Quando è stata l'ultima volta che John Doe ha comprato qualcosa da noi?

AI: John ha comprato per l'ultima volta un cappello Fruity Fedora da noi due settimane fa, il 3 gennaio 2030.

Utente: E Emily Doe?

L'ultima domanda, "E Emily Doe?", è ambigua senza *context*. Se usi questa *query verbatim* per recuperare documenti, probabilmente otterrai risultati irrilevanti. Devi riscrivere questa *query* per riflettere ciò che l'utente sta effettivamente chiedendo. La nuova *query* dovrebbe avere senso da sola. In questo caso, la *query* dovrebbe essere riscritta come "Quando è stata l'ultima volta che Emily Doe ha comprato qualcosa da noi?"

Anche se ho messo la *query rewriting* in "RAG" a pagina 253, la *query rewriting* non è unica per RAG. Nei motori di ricerca tradizionali, la *query rewriting* è spesso fatta usando *euristiche*. Nelle applicazioni AI, la *query rewriting* può anche essere fatta usando altri modelli AI, usando un *prompt* simile a "Data la seguente conversazione, riscrivi l'ultimo *input* dell'utente per riflettere ciò che l'utente sta effettivamente chiedendo". La Figura 6-4 mostra come ChatGPT ha riscritto la *query* usando questo *prompt*.



[Figura 6-4: Puoi usare altri modelli generativi per riscrivere le *query*.]

La *query rewriting* può diventare complicata, specialmente se è necessario fare la *identity resolution* o incorporare altre conoscenze. Ad esempio, se l'utente chiede "E sua moglie?" dovrai prima interrogare il tuo *database* per scoprire chi è sua moglie. Se non hai queste informazioni, il modello di riscrittura dovrebbe riconoscere che questa *query* non è risolvibile invece di *hallucinare* un nome, portando a una risposta sbagliata.

Retrieval contestuale

L'idea alla base del *contextual retrieval* è arricchire ogni *chunk* con un *context* rilevante per renderne più facile il recupero. Una tecnica semplice è arricchire un *chunk* con *metadata* come *tag* e parole chiave. Per l'e-commerce, un prodotto può essere arricchito dalla sua descrizione e recensioni. Immagini e video possono essere interrogati per i loro titoli o didascalie.

I *metadata* possono anche includere entità estratte automaticamente dal *chunk*. Se il tuo documento contiene termini specifici come il codice di errore *EADDRNOTAVAIL* (99), aggiungerli ai *metadata* del documento permette al sistema di recuperarlo con quella parola chiave, anche dopo che il documento è stato convertito in *embedding*.

Puoi anche arricchire ogni *chunk* con le domande a cui può rispondere. Per il supporto clienti, puoi arricchire ogni articolo con domande correlate. Ad esempio, l'articolo su come reimpostare la password può essere arricchito con *query* come "Come reimpostare la password?", "Ho dimenticato la password", "Non riesco ad accedere" o anche "Aiuto, non trovo il mio account".⁹

Se un documento è diviso in più *chunk*, alcuni *chunk* potrebbero mancare del *context* necessario per aiutare il *retriever* a capire di cosa tratta il *chunk*. Per evitare ciò, puoi arricchire ogni *chunk* con il *context* del documento originale, come il titolo e il riassunto del documento originale. Anthropic ha utilizzato modelli *AI* per generare un breve *context*, di solito 50-100 *token*, che spiega il *chunk* e la sua relazione con il documento originale. Ecco il *prompt* utilizzato da Anthropic per questo scopo (Anthropic, 2024):

```
<document>
```

```
{{WHOLE_DOCUMENT}}
```

```
</document>
```

```
Here is the chunk we want to situate within the whole document:
```

```
<chunk>
```

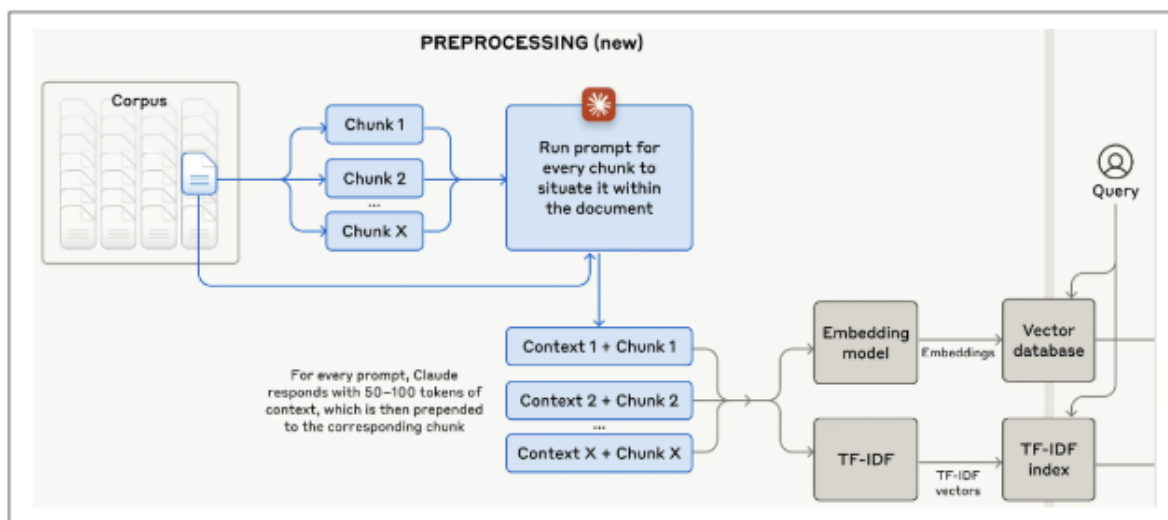
```
{{CHUNK_CONTENT}}
```

```
</chunk>
```

```
Please give a short succinct context to situate this chunk within the overall document for the purposes of improving search retrieval of the chunk. Answer only with the succinct context and nothing else.
```

⁹ Alcuni team mi hanno detto che i loro sistemi di *retrieval* funzionano meglio quando i dati sono organizzati in un formato domanda-risposta.

Il *context* generato per ogni *chunk* è anteposto a ogni *chunk*, e il *chunk* arricchito viene poi *indicizzato* dall'algoritmo di *retrieval*. La Figura 6-5 visualizza il processo che Anthropic segue.



[Figura 6-5: Anthropic arricchisce ogni *chunk* con un breve *context* che situa questo *chunk* all'interno del documento originale, rendendo più facile per il *retriever* trovare i *chunk* rilevanti data una *query*. Immagine da "Introducing Contextual Retrieval" (Anthropic, 2024).]

Valutazione delle Soluzioni di Retrieval

Ecco alcuni fattori chiave da tenere a mente quando si valuta una soluzione di *retrieval*:

- Quali meccanismi di *retrieval* supporta? Supporta la *hybrid search*?
- Se è un *vector database*, quali modelli di *embedding* e algoritmi di *vector search* supporta?
- Quanto è scalabile, sia in termini di archiviazione dei dati che di traffico di *query*? Funziona per i tuoi modelli di traffico?
- Quanto tempo ci vuole per *indicizzare* i tuoi dati? Quanti dati puoi elaborare (come aggiungere/cancellare) in blocco alla volta?
- Qual è la sua *latency* di *query* per diversi algoritmi di *retrieval*?
- Se è una soluzione gestita, qual è la sua struttura dei prezzi? È basata sul volume di documenti/vettori o sul volume di *query*?

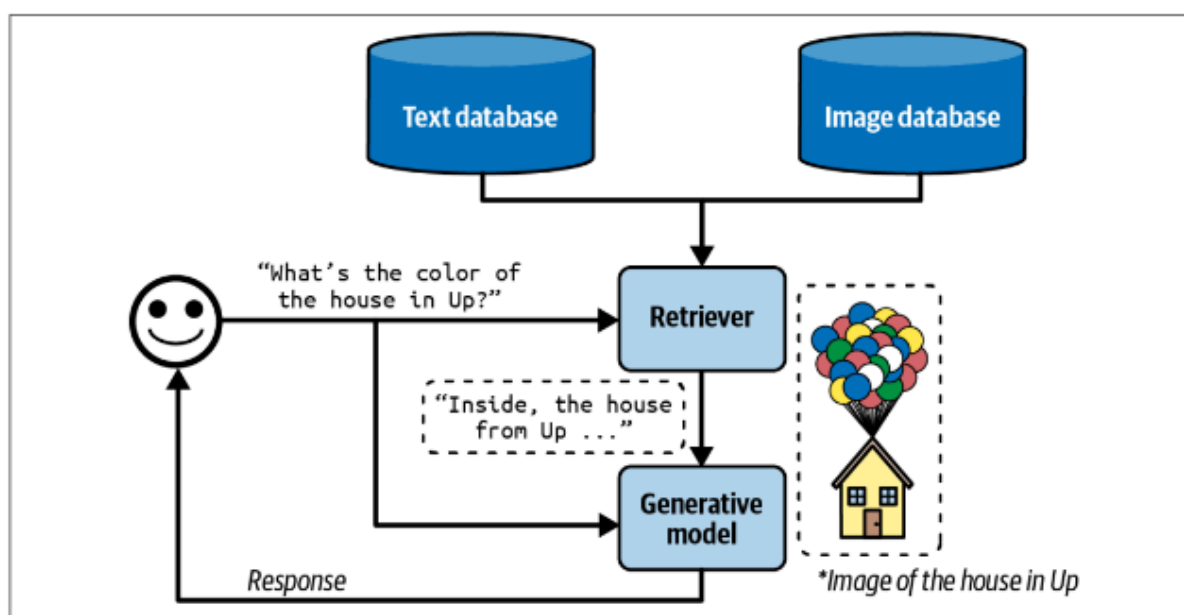
Questo elenco non include le funzionalità tipicamente associate alle soluzioni *enterprise* come il controllo degli accessi, la conformità, la separazione del *data plane* e del *control plane*, ecc.

RAG Oltre i Testi

L'ultima sezione ha discusso i sistemi *RAG basati su testo* dove le fonti di dati esterne sono documenti di testo. Tuttavia, le fonti di dati esterne possono anche essere dati *multimodali* e *tabulari*.

RAG Multimodale

Se il tuo *generator* è *multimodale*, i suoi *context* potrebbero essere arricchiti non solo con documenti di testo ma anche con immagini, video, audio, ecc., da fonti esterne. Userò immagini negli esempi per mantenere la scrittura concisa, ma puoi sostituire le immagini con qualsiasi altra modalità. Data una *query*, il *retriever* recupera sia testi che immagini rilevanti per essa. Ad esempio, data "Di che colore è la casa nel film Pixar Up?" il *retriever* può recuperare un'immagine della casa in *Up* per aiutare il modello a rispondere, come mostrato nella Figura 6-6.



[Figura 6-6: Il *RAG multimodale* può arricchire una *query* con sia testo che immagini. (*L'immagine reale da *Up* non è usata, per motivi di copyright.)]

Se le immagini hanno *metadata*—come titoli, *tag* e didascalie—possono essere recuperate usando i *metadata*. Ad esempio, un'immagine viene recuperata se la sua didascalia è considerata rilevante per la *query*.

Se vuoi recuperare immagini in base al loro contenuto, avrai bisogno di un modo per confrontare le immagini con le *query*. Se le *query* sono testi, avrai bisogno di un modello di *embedding multimodale* che possa generare *embedding* sia per immagini che per testi. Diciamo che usi *CLIP* (Radford et al., 2021) come modello di *embedding multimodale*. Il *retriever* funziona come segue:

1. Genera *embedding CLIP* per tutti i tuoi dati, sia testi che immagini, e memorizzali in un *vector database*.
2. Data una *query*, genera il suo *embedding CLIP*.
3. Interroga il *vector database* per tutte le immagini e i testi i cui *embedding* sono vicini all'*embedding* della *query*.

RAG con dati tabulari

La maggior parte delle applicazioni lavora non solo con dati *non strutturati* come testi e immagini ma anche con dati *tabulari*. Molte *query* potrebbero aver bisogno di informazioni da tabelle di dati per rispondere. Il flusso di lavoro per arricchire un *context* usando dati tabulari è significativamente diverso dal classico flusso di lavoro RAG.

Immagina di lavorare per un sito *e-commerce* chiamato Kitty Vogue che si specializza in moda per gatti. Questo negozio ha una tabella ordini chiamata Vendite, come mostrato nella Tabella 6-3.

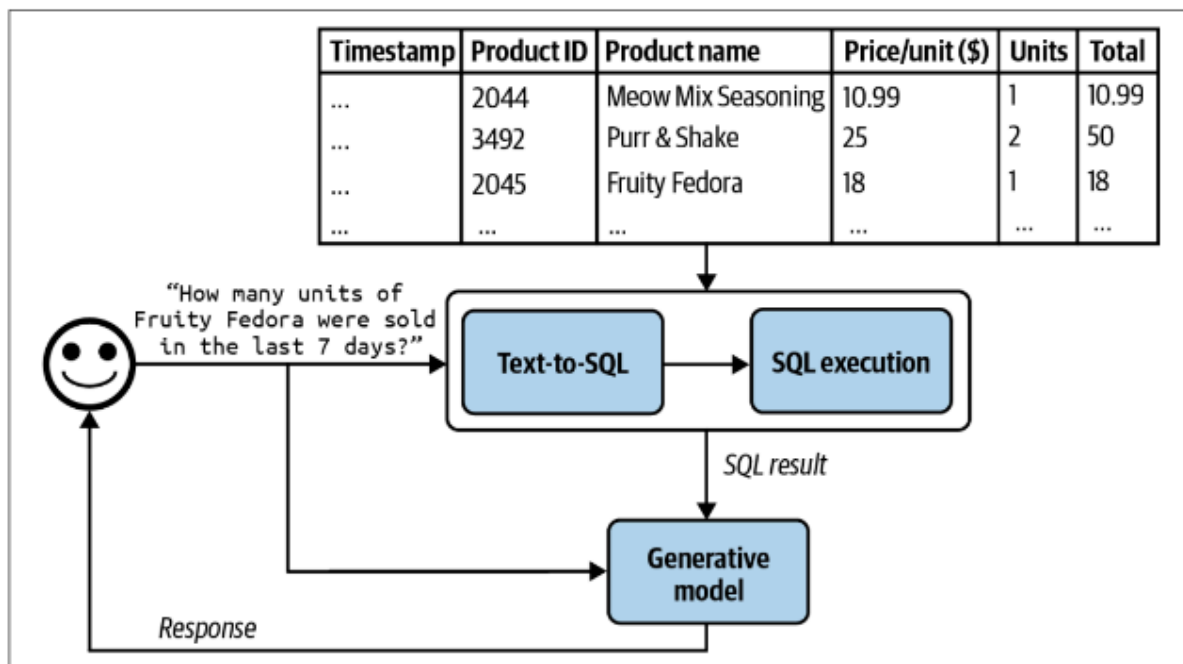
ID	Timestamp	ID Prod.	Prodotto	Prezzo unitario (\$)	Unità	Totale
1	...	2044	Meow Mix Seasoning	10.99	1	10.99
2	...	3492	Purr & Shake	25	2	50
3	...	2045	Fruity Fedora	18	1	18
...	...	...	...	...	...	...

Per generare una risposta alla domanda "Quante unità di Fruity Fedora sono state vendute negli ultimi 7 giorni?", il tuo sistema deve interrogare questa tabella per tutti gli ordini che coinvolgono Fruity Fedora e sommare il numero di unità in tutti gli ordini. Supponi che questa tabella possa essere interrogata usando SQL. La *query SQL* potrebbe assomigliare a questa:

```
SELECT SUM(units) AS total_units_sold
FROM Sales
WHERE product_name = 'Fruity Fedora'
AND timestamp >= DATE_SUB(CURDATE(), INTERVAL 7 DAY);
```

Il flusso di lavoro è il seguente, visualizzato nella Figura 6-7. Per eseguire questo flusso di lavoro, il tuo sistema deve avere la capacità di generare ed eseguire la *query SQL*:

1. Text-to-SQL: in base alla *query* dell'utente e agli *schema* di tabella forniti, determina quale *query SQL* è necessaria. *Text-to-SQL* è un esempio di *semantic parsing*, come discusso nel Capitolo 2.
2. Esecuzione SQL: esegui la *query SQL*.
3. Generazione: genera una risposta basata sul risultato SQL e sulla *query* utente originale.



[Figura 6-7: Un sistema RAG che arricchisce il *context* con dati tabulari.]

Per il passo *text-to-SQL*, se ci sono molte tabelle disponibili i cui *schema* non possono essere inseriti tutti nel *context* del modello, potresti aver bisogno di un passo intermedio per prevedere quali tabelle usare per ogni *query*. *Text-to-SQL* può essere fatto dallo stesso *generator* che genera la risposta finale o da un modello *text-to-SQL* specializzato.

In questa sezione, abbiamo discusso come strumenti come i *retriever* e gli *SQL executor* possono permettere ai modelli di gestire più *query* e generare risposte di qualità superiore. Dare a un modello accesso a più strumenti migliorerebbe ancora di più le sue capacità? L'uso di strumenti è una caratteristica fondamentale del modello *agentic*, che discuteremo nella prossima sezione.

Agenti

Gli *agent intelligenti* sono considerati da molti l'obiettivo ultimo dell'IA. Il libro classico di Stuart Russell e Peter Norvig, *Artificial Intelligence: A Modern Approach* (Prentice Hall, 1995) definisce il campo della ricerca sull'intelligenza artificiale come "lo studio e la progettazione di *agent razionali*".

Le capacità senza precedenti dei *foundation model* hanno aperto la porta ad applicazioni *agentic* che prima erano inimmaginabili. Queste nuove capacità rendono finalmente possibile sviluppare *agent* autonomi e intelligenti per agire come nostri assistenti, colleghi e *coach*. Possono aiutarci a creare un sito web, raccogliere dati, pianificare un viaggio, fare ricerche di mercato, gestire un account cliente, automatizzare l'inserimento di dati, prepararci per i colloqui, intervistare i nostri candidati, negoziare un affare, ecc. Le possibilità sembrano infinite e il potenziale valore economico di questi *agent* è enorme.

Gli *agent* alimentati da *IA* sono un campo emergente, senza *framework* teorici stabiliti per definirli, svilupparli e valutarli. Questa sezione è un tentativo del meglio possibile di costruire un *framework* dalla letteratura esistente, ma si evolverà man mano che il campo si evolve. Rispetto al resto del libro, questa sezione è più sperimentale.

Questa sezione inizierà con una panoramica degli *agent*, e poi continuerà con due aspetti che determinano le capacità di un *agent*: *tool* e *planning*. Gli *agent*, con i loro nuovi modi di operare, hanno nuovi modi di fallire. Questa sezione si concluderà con una discussione su come valutare gli *agent* per catturare questi fallimenti.

Anche se gli *agent* sono nuovi, sono costruiti su concetti che sono già apparsi in questo libro, inclusi l'*auto-critica*, la *chain-of-thought* e gli *output strutturati*.

Panoramica sugli Agenti

Il termine *agent* è stato usato in molti contesti ingegneristici diversi, inclusi ma non limitati a *software agent*, *intelligent agent*, *user agent*, *conversational agent* e *reinforcement learning agent*. Quindi, cos'è esattamente un *agent*?

Un *agent* è qualsiasi cosa che possa percepire il suo *environment* e agire su quell'*environment*.¹⁰ Ciò significa che un *agent* è caratterizzato dall'*environment* in cui opera e dall'*insieme di azioni* che può eseguire.

L'*environment* in cui un *agent* può operare è definito dal suo caso d'uso. Se un *agent* è sviluppato per giocare a un gioco (ad es., *Minecraft*, *Go*, *Dota*), quel gioco è il suo *environment*. Se vuoi che un *agent* scarichi documenti da *internet*, l'*environment* è *internet*. Se il tuo *agent* è un robot da cucina, la cucina è il suo *environment*. L'*environment* di un *agent* di auto a guida autonoma è il sistema stradale e le aree adiacenti.

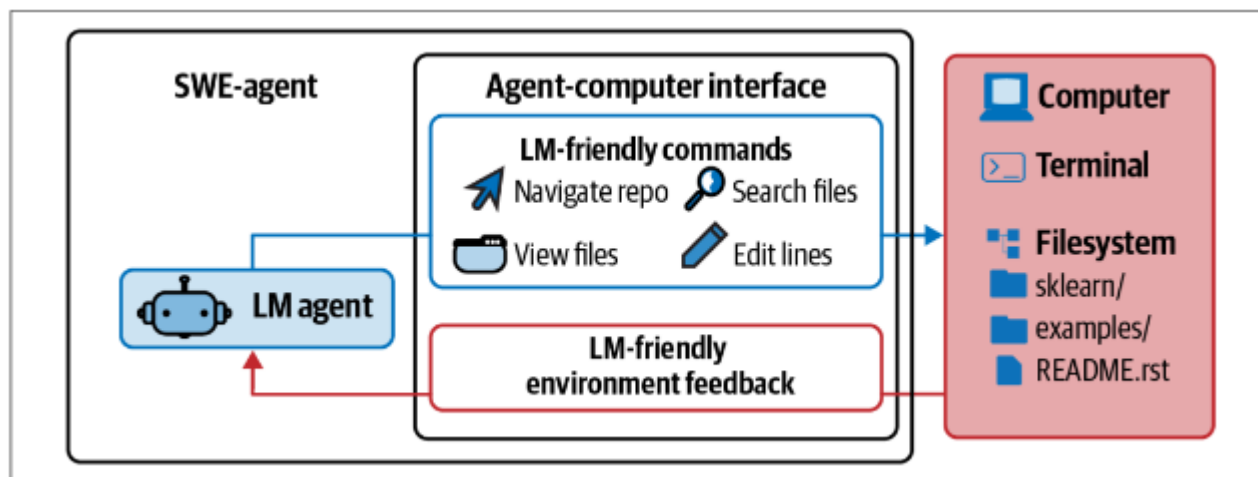
L'*insieme di azioni* che un *agent* di *IA* può eseguire è aumentato dagli *tool* a cui ha accesso. Molte applicazioni alimentate da *AI generativa* con cui interagisci quotidianamente sono *agent* con accesso a *tool*, sebbene semplici. ChatGPT è un *agent*. Può cercare sul *web*, eseguire codice *Python* e generare immagini. I sistemi *RAG* sono *agent*, e i *retriever* di testo, i *retriever* di immagini e gli *SQL executor* sono i loro *tool*.

C'è una forte dipendenza tra l'*environment* di un *agent* e il suo insieme di *tool*. L'*environment* determina quali *tool* un *agent* può potenzialmente usare. Ad esempio, se l'*environment* è una partita a scacchi, le uniche azioni possibili per un *agent* sono le mosse di scacchi valide. Tuttavia, l'*inventario* degli *tool* di un *agent* restringe l'*environment* in cui può operare.

10: *Artificial Intelligence: A Modern Approach* (1995) definisce un *agent* come qualsiasi cosa che possa essere vista come percepire il suo *environment* attraverso *sensor* e agire su quell'*environment* attraverso *actuator*.

in. Ad esempio, se l'unica azione di un robot è nuotare, sarà confinato a un *environment* acquatico.

La Figura 6-8 mostra una visualizzazione di *SWE-agent* (Yang et al., 2024), un *agent* costruito su *GPT-4*. Il suo *environment* è il computer con il *terminal* e il *file system*. Il suo insieme di azioni include *navigate repo*, *search files*, *view files* e *edit lines*.



[Figura 6-8: *SWE-agent* (Yang et al., 2024) è un *coding agent* il cui *environment* è il computer e le cui azioni includono navigazione, ricerca e modifica. Adattato da un'immagine originale concessa in licenza CC BY 4.0.]

Un *agent* di IA è destinato a compiere *task* tipicamente forniti dagli utenti negli *input*. In un *agent* di IA, l'IA è il cervello che elabora le informazioni che riceve, incluso il *task* e il *feedback* dall'*environment*, pianifica una sequenza di azioni per raggiungere questo *task* e determina se il *task* è stato realizzato.

Torniamo al sistema RAG con dati tabulari nell'esempio di Kitty Vogue. Questo è un *agent* semplice con tre azioni: *response generation*, *SQL query generation* e *SQL query execution*. Data la *query* "Proietta le entrate delle vendite per Fruity Fedora nei prossimi tre mesi", l'*agent* potrebbe eseguire la seguente sequenza di azioni:

1. Ragionare su come realizzare questo *task*. Potrebbe decidere che per prevedere le vendite future, ha prima bisogno dei numeri di vendita degli ultimi cinque anni. Nota che il ragionamento dell'*agent* è mostrato come sua risposta intermedia.
2. Invocare la *SQL query generation* per generare la *query* per ottenere i numeri di vendita degli ultimi cinque anni.
3. Invocare la *SQL query execution* per eseguire questa *query*.
4. Ragionare sugli *output* degli *tool* e su come aiutano con la previsione delle vendite. Potrebbe decidere che questi numeri sono insufficienti per fare una proiezione affidabile, forse a causa di valori mancanti. Poi decide che ha anche bisogno di informazioni sulle campagne di *marketing* passate.

5. Invocare la *SQL query generation* per generare le *query* per le campagne di *marketing* passate.
6. Invocare la *SQL query execution*.
7. Ragionare che questa nuova informazione è sufficiente per aiutare a prevedere le vendite future. Poi genera una proiezione.
8. Ragionare che il *task* è stato completato con successo.

Rispetto ai casi d'uso non *agentici*, gli *agent* tipicamente richiedono modelli più potenti per due ragioni:

- Errori composti: un *agent* spesso ha bisogno di eseguire più passi per realizzare un *task*, e l'*accuratezza* complessiva diminuisce all'aumentare del numero di passi. Se l'*accuratezza* del modello è del 95% per passo, su 10 passi, l'*accuratezza* scenderà al 60% e su 100 passi, l'*accuratezza* sarà solo dello 0,6%.
- Posta in gioco più alta: con l'accesso agli *tool*, gli *agent* sono capaci di eseguire *task* più impattanti, ma qualsiasi fallimento potrebbe avere conseguenze più gravi.

Un *task* che richiede molti passi può richiedere tempo e denaro per essere eseguito. Tuttavia, se gli *agent* possono essere autonomi, possono risparmiare molto tempo umano, rendendo i loro costi vantaggiosi.

Dato un *environment*, il successo di un *agent* in un *environment* dipende dall'*inventario* degli *tool* a cui ha accesso e dalla forza del suo *AI planner*. Iniziamo guardando i diversi tipi di *tool* che un modello può usare.

Strumenti

Un sistema non ha bisogno di accesso a *tool* esterni per essere un *agent*. Tuttavia, senza *tool* esterni, le capacità dell'*agent* sarebbero limitate. Di per sé, un modello può tipicamente eseguire un'azione—ad esempio, un *LLM* può generare testo e un *image generator* può generare immagini. I *tool* esterni rendono un *agent* molto più capace.

I *tool* aiutano un *agent* sia a percepire l'*environment* che ad agire su di esso. Le azioni che permettono a un *agent* di percepire l'*environment* sono azioni di *sola lettura*, mentre le azioni che permettono a un *agent* di agire sull'*environment* sono azioni di *scrittura*.

Questa sezione fornisce una panoramica dei *tool* esterni. Come i *tool* possono essere usati sarà discusso in "*Planning*" a pagina 281.

L'insieme di *tool* a cui un *agent* ha accesso è il suo *tool inventory*. Poiché il *tool inventory* di un *agent* determina cosa un *agent* può fare, è importante pensare a cosa e

¹¹ Una lamentela nei primi giorni degli *agent* è che gli *agent* sono solo bravi a bruciare i tuoi crediti *API*.

a quanti *tool* dare a un *agent*. Più *tool* danno a un *agent* più capacità. Tuttavia, più *tool* ci sono, più è difficile comprenderli e utilizzarli bene. È necessaria la sperimentazione per trovare il giusto insieme di *tool*, come discusso in "*Tool selection*" a pagina 295.

A seconda dell'*environment* dell'*agent*, ci sono molti *tool* possibili. Ecco tre categorie di *tool* che potresti voler considerare: *knowledge augmentation* (cioè costruzione del *context*), *capability extension* e *tool* che permettono al tuo *agent* di agire sul suo *environment*.

Knowledge augmentation

Spero che questo libro, finora, ti abbia convinto dell'importanza di avere il *context* rilevante per la qualità della risposta di un modello. Un'importante categoria di *tool* include quelli che aiutano ad arricchire la conoscenza del tuo *agent*. Alcuni di essi sono già stati discussi: *text retriever*, *image retriever* e *SQL executor*. Altri *tool* potenziali includono *internal people search*, un'*inventory API* che restituisce lo stato di diversi prodotti, *Slack retrieval*, un *email reader*, ecc.

Molti di questi *tool* arricchiscono un modello con i processi e le informazioni privati della tua organizzazione. Tuttavia, i *tool* possono anche dare ai modelli accesso a informazioni pubbliche, specialmente da *internet*.

Il *web browsing* è stata tra le prime e più anticipate capacità da incorporare in *chatbot* come ChatGPT. Il *web browsing* previene che un modello diventi *stale*. Un modello diventa *stale* quando i dati su cui è stato *addestrato* diventano obsoleti. Se i dati di *training* del modello sono stati interrotti la scorsa settimana, non sarà in grado di rispondere a domande che richiedono informazioni di questa settimana a meno che queste informazioni non siano fornite nel *context*. Senza *web browsing*, un modello non sarà in grado di dirti sul meteo, notizie, eventi imminenti, prezzi delle azioni, stato dei voli, ecc.

Uso *web browsing* come termine ombrello per coprire tutti i *tool* che accedono a *internet*, inclusi *web browser* e *API* specifiche come *search API*, *news API*, *GitHub API* o *social media API* come quelli di X, LinkedIn e Reddit.

Mentre il *web browsing* permette al tuo *agent* di fare riferimento a informazioni aggiornate per generare risposte migliori e ridurre le *hallucinations*, può anche aprire il tuo *agent* alle porcherie di *internet*. Seleziona le tue *Internet API* con cura.

Capability extension

La seconda categoria di *tool* da considerare sono quelli che affrontano le limitazioni intrinseche dei modelli di *IA*. Sono modi facili per dare al tuo modello una spinta alle prestazioni. Ad esempio, i modelli di *IA* sono notoriamente pessimi in matematica. Se chiedi a un modello quanto fa 199.999 diviso 292, il modello probabilmente fallirà. Tuttavia, questo calcolo è banale se

il modello ha accesso a una calcolatrice. Invece di cercare di *addestrare* il modello a essere bravo in aritmetica, è molto più efficiente in termini di risorse dare semplicemente al modello accesso a un *tool*. Altri *tool* semplici che possono aumentare significativamente la capacità di un modello includono un calendario, un *timezone converter*, un *unit converter* (ad es., da libbre a chilogrammi) e un *translator* che può tradurre da e verso le lingue in cui il modello non è bravo.

Tool più complessi ma potenti sono gli *code interpreter*. Invece di *addestrare* un modello a comprendere il codice, puoi dargli accesso a un *code interpreter* in modo che possa eseguire un pezzo di codice, restituire i risultati o analizzare i fallimenti del codice. Questa capacità permette ai tuoi *agent* di agire come *coding assistant*, *data analyst* e persino *research assistant* che possono scrivere codice per eseguire esperimenti e riportare risultati. Tuttavia, l'esecuzione automatica del codice comporta il rischio di *code injection attacks*, come discusso in "*Defensive Prompt Engineering*" a pagina 235. Misure di sicurezza appropriate sono cruciali per mantenere te e i tuoi utenti al sicuro.

I *tool* esterni possono rendere *multimodale* un modello solo testuale o solo visivo. Ad esempio, un modello che può generare solo testi può sfruttare un modello *text-to-image* come *tool*, permettendogli di generare sia testi che immagini. Data una richiesta di testo, l'*AI planner* dell'*agent* decide se invocare la *text generation*, la *image generation* o entrambe. È così che ChatGPT può generare sia testo che immagini—usa DALL-E come suo *image generator*. Gli *agent* possono anche usare un *code interpreter* per generare grafici e diagrammi, un compilatore *LaTeX* per renderizzare equazioni matematiche o un *browser* per renderizzare pagine web da codice *HTML*.

Allo stesso modo, un modello che può elaborare solo *input* testuali può usare uno *tool* di *image captioning* per elaborare immagini e uno *tool* di *transcription* per elaborare audio. Può usare uno *tool* *OCR* (*optical character recognition*) per leggere *PDF*.

L'uso di *tool* può aumentare significativamente le prestazioni di un modello rispetto al solo *prompting* o persino al *finetuning*. Chameleon (Lu et al., 2023) mostra che un *agent* alimentato da *GPT-4*, arricchito con un insieme di 13 *tool*, può superare *GPT-4* da solo su diversi *benchmark*. Esempi di *tool* usati da questo *agent* sono *knowledge retrieval*, un *query generator*, un *image captioner*, un *text detector* e *Bing search*.

Su *ScienceQA*, un *benchmark* di risposta a domande scientifiche, Chameleon migliora il miglior risultato *few-shot* pubblicato dell'11,37%. Su *TabMWP* (*Tabular Math Word Problems*) (Lu et al., 2022), un *benchmark* che coinvolge domande matematiche tabellari, Chameleon migliora l'accuratezza del 17%.

Write actions

Finora, abbiamo discusso azioni di *sola lettura* che permettono a un modello di leggere dalle sue fonti di dati. Ma i *tool* possono anche eseguire azioni di *scrittura*, apportando modifiche alle fonti di dati. Un *SQL executor* può recuperare una tabella di dati (*lettura*) ma può anche cambiare o cancellare la tabella

(scrittura). Un'email API può leggere un'email ma può anche rispondere ad essa. Una *banking API* può recuperare il tuo saldo corrente ma può anche avviare un bonifico bancario.

Le azioni di *scrittura* permettono a un sistema di fare di più. Possono permetterti di automatizzare l'intero flusso di lavoro di *customer outreach*: ricerca di potenziali clienti, ricerca dei loro contatti, stesura di email, invio delle prime email, lettura delle risposte, *follow-up*, estrazione degli ordini, aggiornamento dei tuoi *database* con nuovi ordini, ecc.

Tuttavia, la prospettiva di dare all'IA la capacità di alterare automaticamente le nostre vite è spaventosa. Proprio come non dovresti dare a un tirocinante l'autorità di cancellare il tuo *production database*, non dovresti permettere a un'IA inaffidabile di avviare bonifici bancari. La fiducia nelle capacità del sistema e nelle sue misure di sicurezza è cruciale. Devi assicurarti che il sistema sia protetto da *bad actor* che potrebbero tentare di manipolarlo per eseguire azioni dannose.

Quando parlo di *agent* di IA autonomi a un gruppo di persone, c'è spesso qualcuno che tira fuori l'esempio delle auto a guida autonoma. "E se qualcuno si introducesse nell'auto per rapirti?" Sebbene l'esempio dell'auto a guida autonoma sembri viscerale a causa della sua fisicità, un sistema di IA può causare danni senza una presenza nel mondo fisico. Può manipolare il mercato azionario, rubare *copyright*, violare la privacy, rafforzare *bias*, diffondere *misinformation* e propaganda, e altro, come discusso in "*Defensive Prompt Engineering*" a pagina 235.

Queste sono tutte preoccupazioni valide, e qualsiasi organizzazione che voglia sfruttare l'IA deve prendere sul serio sicurezza e protezione. Tuttavia, questo non significa che ai sistemi di IA non dovrebbe mai essere data la capacità di agire nel mondo reale. Se possiamo convincere le persone a fidarsi di una macchina per portarci nello spazio, spero che un giorno le misure di sicurezza saranno sufficienti per farci fidare dei sistemi di IA autonomi. Inoltre, anche gli umani possono fallire. Personalmente, mi fiderei di più di un'auto a guida autonoma che della persona media per farmi guidare in giro.

Proprio come i *tool* giusti possono aiutare gli umani a essere molto più produttivi—riesci a immaginare di fare affari senza Excel o costruire un grattacielo senza gru?—i *tool* permettono ai modelli di realizzare molti più *task*. Molti fornitori di modelli supportano già l'uso di *tool* con i loro modelli, una funzionalità spesso chiamata *function calling*. Andando avanti, mi aspetto che il *function calling* con un'ampia serie di *tool* sia comune con la maggior parte dei modelli.

Planning

Al centro di un *foundation model agent* c'è il modello responsabile di risolvere un *task*. Un *task* è definito dal suo *goal* e dai suoi *constraint*. Ad esempio, un *task* è pianificare un viaggio di due settimane da San Francisco all'India con un *budget* di \$5.000. Il *goal* è il viaggio di due settimane. Il *constraint* è il *budget*.

Task complessi richiedono *planning*. L'*output* del processo di *planning* è un *plan*, che è una *roadmap* che delinea i passi necessari per realizzare un *task*. Un *planning* efficace richiede tipicamente che il modello comprenda il *task*, consideri diverse opzioni per realizzare questo *task* e scelga quella più promettente.

Se sei mai stato in una riunione di *planning*, sai che *planning* è difficile. Come importante problema computazionale, il *planning* è ben studiato e richiederebbe diversi volumi per essere coperto. Sarò in grado di coprire solo la superficie qui.

Panoramica sul Planning

Dato un *task*, ci sono molti modi possibili per scomporlo, ma non tutti porteranno a un esito di successo. Tra le soluzioni corrette, alcune sono più efficienti di altre. Considera la *query*, "Quante aziende senza *revenue* hanno raccolto almeno 1 miliardo di dollari?" Ci sono molti modi possibili per risolverlo, ma come illustrazione, considera le due opzioni:

1. Trova tutte le aziende senza *revenue*, poi filtra quelle per l'importo raccolto.
2. Trova tutte le aziende che hanno raccolto almeno 1 miliardo di dollari, poi filtra quelle per *revenue*.

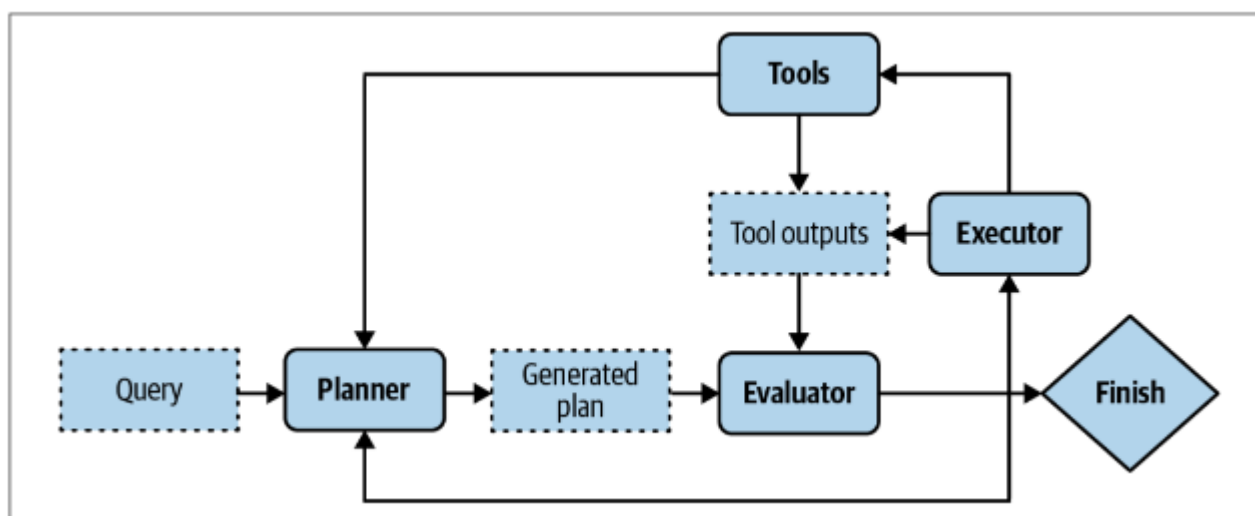
La seconda opzione è più efficiente. Ci sono molte più aziende senza *revenue* che aziende che hanno raccolto 1 miliardo di dollari. Date solo queste due opzioni, un *agent intelligente* dovrebbe scegliere l'opzione 2.

Puoi accoppiare il *planning* con l'*execution* nello stesso *prompt*. Ad esempio, dai al modello un *prompt*, chiedigli di pensare passo dopo passo (come con un *chain-of-thought prompt*), e poi esegui quei passi tutti in un unico *prompt*. Ma cosa succede se il modello elabora un *plan* di 1.000 passi che non realizza nemmeno l'obiettivo? Senza supervisione, un *agent* può eseguire quei passi per ore, sprecando tempo e denaro in chiamate *API*, prima che tu ti renda conto che non sta andando da nessuna parte.

Per evitare un'esecuzione infruttuosa, il *planning* dovrebbe essere disaccoppiato dall'*execution*. Chiedi all'*agent* di generare prima un *plan*, e solo dopo che questo *plan* è *validato* viene eseguito. Il *plan* può essere *validato* usando *euristiche*. Ad esempio, una semplice *euristica* è eliminare i *plan* con azioni non valide. Se il *plan* generato richiede una *Google search* e l'*agent* non ha accesso a *Google Search*, questo *plan* non è valido. Un'altra semplice *euristica* potrebbe essere eliminare tutti i *plan* con più di X passi. Un *plan* può anche essere *validato* usando *AI judges*. Puoi chiedere a un modello di valutare se il *plan* sembra ragionevole o come migliorarlo.

Se il *plan* generato è valutato come cattivo, puoi chiedere al *planner* di generare un altro *plan*. Se il *plan* generato è buono, eseguillo. Se il *plan* consiste di *tool* esterni, verrà invocato il *function calling*. Gli *output* dall'esecuzione di questo *plan* dovranno poi essere nuovamente valutati. Nota che il *plan* generato non deve essere un

plan end-to-end per l'intero *task*. Può essere un piccolo *plan* per un *subtask*. L'intero processo appare come nella Figura 6-9.



[Figura 6-9: Disaccoppiamento di *planning* e *execution* in modo che solo i *plan* validati vengano eseguiti.]

Il tuo sistema ora ha tre componenti: uno per generare *plan*, uno per validare i *plan* e un altro per eseguire i *plan*. Se consideri ogni componente un *agent*, questo è un *multi-agent system*.¹²

Per accelerare il processo, invece di generare *plan* in sequenza, puoi generare diversi *plan* in parallelo e chiedere all'*evaluator* di scegliere il più promettente. Questo è un altro compromesso *latency/costo*, poiché generare più *plan* simultaneamente comporterà costi extra.

Il *planning* richiede la comprensione dell'*intention* dietro un *task*: cosa sta cercando di fare l'utente con questa *query*? Un *intent classifier* è spesso usato per aiutare gli *agent* a pianificare. Come mostrato in "Scomporre Task Complessi in *Subtask* più Semplici" a pagina 224, la *intent classification* può essere fatta usando un altro *prompt* o un modello di classificazione addestrato per questo *task*. Il meccanismo di *intent classification* può essere considerato un altro *agent* nel tuo *multi-agent system*.

Conoscere l'*intent* può aiutare l'*agent* a scegliere i *tool* giusti. Ad esempio, per il supporto clienti, se la *query* riguarda la fatturazione, l'*agent* potrebbe aver bisogno di accedere a uno *tool* per recuperare i pagamenti recenti di un utente. Ma se la *query* riguarda come reimpostare una password, l'*agent* potrebbe aver bisogno di accedere al *documentation retrieval*.

¹² Poiché la maggior parte dei flussi di lavoro *agentic* sono sufficientemente complessi da coinvolgere più componenti, la maggior parte degli *agent* sono *multi-agent*.

Alcune *query* potrebbero essere fuori dallo *scope* dell'*agent*. L'*intent classifier* dovrebbe essere in grado di classificare le richieste come *IRRELEVANT* in modo che l'*agent* possa rifiutarle educatamente invece di sprecare *FLOP* cercando di trovare soluzioni impossibili.

Finora, abbiamo assunto che l'*agent* automatizzi tutti e tre gli stadi: generare *plan*, validare i *plan* ed eseguire i *plan*. In realtà, gli umani possono essere coinvolti in qualsiasi di questi stadi per aiutare con il processo e mitigare i rischi. Un esperto umano può fornire un *plan*, validare un *plan* o eseguire parti di un *plan*. Ad esempio, per *task* complessi per cui un *agent* ha difficoltà a generare l'intero *plan*, un esperto umano può fornire un *plan* di alto livello che l'*agent* può espandere. Se un *plan* coinvolge operazioni rischiose, come aggiornare un *database* o unire una modifica del codice, il sistema può chiedere l'approvazione umana esplicita prima dell'esecuzione o lasciare che gli umani eseguano queste operazioni. Per rendere ciò possibile, è necessario definire chiaramente il livello di automazione che un *agent* può avere per ogni azione.

Per riassumere, risolvere un *task* coinvolge tipicamente i seguenti processi. Nota che la *reflection* non è obbligatoria per un *agent*, ma aumenterà significativamente le prestazioni dell'*agent*:

1. Plan generation: elaborare un *plan* per realizzare questo *task*. Un *plan* è una sequenza di azioni gestibili, quindi questo processo è anche chiamato *task decomposition*.
2. Reflection and error correction: valutare il *plan* generato. Se è un *plan* cattivo, generane uno nuovo.
3. Execution: eseguire le azioni delineate nel *plan* generato. Questo spesso coinvolge chiamare *function* specifiche.
4. Reflection and error correction: dopo aver ricevuto gli esiti delle azioni, valutare questi esiti e determinare se l'obiettivo è stato realizzato. Identificare e correggere gli errori. Se l'obiettivo non è completato, generare un nuovo *plan*.

Hai già visto alcune tecniche per la *plan generation* e la *reflection* in questo libro. Quando chiedi a un modello di "pensare passo dopo passo", gli stai chiedendo di scomporre un *task*. Quando chiedi a un modello di "verificare se la tua risposta è corretta", gli stai chiedendo di riflettere.

Foundation models as planners

Una domanda aperta è quanto bene i *foundation model* possano pianificare. Molti ricercatori credono che i *foundation model*, almeno quelli costruiti su *autoregressive language model*, non possano. Il Chief AI Scientist di Meta Yann LeCun afferma inequivocabilmente che gli *autoregressive LLM* non possono pianificare (2023). Nell'articolo "Possono gli *LLM* davvero ragionare e pianificare?" Kambhampati (2023) sostiene che gli *LLM* sono eccellenti nell'estrarre conoscenza ma non nel pianificare. Kambhampati suggerisce che gli articoli che affermano le capacità di pianificazione degli *LLM* confondono la conoscenza generale di pianificazione estratta dagli *LLM* con *plan eseguibili*.

"I *plan* che escono dagli *LLM* possono sembrare ragionevoli all'utente comune, e tuttavia portare a interazioni ed errori al momento dell'*execution*."

Tuttavia, mentre ci sono molte prove aneddotiche che gli *LLM* sono scarsi *planner*, non è chiaro se sia perché non sappiamo come usare gli *LLM* nel modo giusto o perché gli *LLM*, fondamentalmente, non possono pianificare.

Il planning, nella sua essenza, è un problema di search. Cercate tra diversi percorsi verso l'obiettivo, prevedete l'esito (*reward*) di ogni percorso e scegliete il percorso con l'esito più promettente. Spesso, potreste determinare che non esiste alcun percorso che possa portarvi all'obiettivo.

La *search* spesso richiede il *backtracking*. Ad esempio, immaginate di essere a un passo in cui ci sono due azioni possibili: A e B. Dopo aver intrapreso l'azione A, entrate in uno *state* che non è promettente, quindi dovete fare *backtracking* allo *state* precedente per intraprendere l'azione B.

Alcune persone sostengono che un modello *autoregressive* può generare solo azioni in avanti. Non può fare *backtracking* per generare azioni alternative. A causa di ciò, concludono che i modelli *autoregressive* non possono pianificare. Tuttavia, questo non è necessariamente vero. Dopo aver eseguito un percorso con l'azione A, se il modello determina che questo percorso non ha senso, può rivedere il percorso usando l'azione B invece, facendo efficacemente *backtracking*. Il modello può anche sempre ricominciare da capo e scegliere un altro percorso.

È anche possibile che gli *LLM* siano scarsi *planner* perché non vengono forniti loro gli *tooling* necessari per pianificare. Per pianificare, è necessario conoscere non solo le azioni disponibili ma anche *il potenziale esito di ogni azione*. Come semplice esempio, supponiamo che vogliate camminare su una montagna. Le vostre azioni potenziali sono girare a destra, girare a sinistra, girarsi o andare dritto. Tuttavia, se girare a destra vi farà cadere da una scogliera, potreste non voler considerare questa azione. In termini tecnici, un'azione vi porta da uno *state* a un altro, ed è necessario conoscere lo *state* risultante per determinare se intraprendere un'azione.

Ciò significa che non è sufficiente *promptare* un modello per generare solo una sequenza di azioni come fa la popolare tecnica del *chain-of-thought prompting*. L'articolo "Ragionare con il *Language Model* è Pianificare con il *World Model*" (Hao et al., 2023) sostiene che un *LLM*, contenendo così tante informazioni sul mondo, è capace di prevedere l'esito di ogni azione. Questo *LLM* può incorporare questa previsione dell'esito per generare *plan* coerenti.

Anche se l'*IA* non può pianificare, può ancora essere parte di un *planner*. Potrebbe essere possibile arricchire un *LLM* con uno *search tool* e un sistema di *state tracking* per aiutarlo a pianificare.

Foundation Model (FM) Versus Reinforcement Learning (RL) Planners

L'*agent* è un concetto centrale nell'*RL*, che è definito su Wikipedia come un campo "preoccupato di come un *agent intelligente* dovrebbe intraprendere azioni in un *environment dinamico* per massimizzare il *reward cumulativo*."

Gli *RL agent* e gli *FM agent* sono simili in molti modi. Sono entrambi caratterizzati dai loro *environment* e dalle possibili azioni. La differenza principale è nel modo in cui funzionano i loro *planner*. In un *RL agent*, il *planner* è *addestrato* da un algoritmo *RL*. Addestrare questo *RL planner* può richiedere molto tempo e risorse. In un *FM agent*, il modello è il *planner*. Questo modello può essere *promptato* o *finetunato* per migliorare le sue capacità di *planning*, e generalmente richiede meno tempo e meno risorse.

Tuttavia, non c'è nulla che impedisca a un *FM agent* di incorporare algoritmi *RL* per migliorare le sue prestazioni. Sospetto che a lungo termine, gli *FM agent* e gli *RL agent* si fonderanno.

Plan generation

Il modo più semplice per trasformare un modello in un *plan generator* è con il *prompt engineering*. Immagina di voler creare un *agent* per aiutare i clienti a conoscere i prodotti di Kitty Vogue. Dai a questo *agent* accesso a tre *tool* esterni: *retrieve products by price*, *retrieve top products* e *retrieve product information*. Ecco un esempio di *prompt* per la *plan generation*. Questo *prompt* è solo a scopo illustrativo. I *prompt* di produzione sono probabilmente più complessi:

SYSTEM PROMPT

Proponi un *plan* per risolvere il *task*. Hai accesso a 5 azioni:

```
get_today_date()
fetch_top_products(start_date, end_date, num_products)
fetch_product_info(product_name)
generate_query(task_history, tool_output)
generate_response(query)
```

Il *plan* deve essere una sequenza di azioni valide.

Esempi

Task: "Tell me about Fruity Fedora"

Plan: [fetch_product_info, generate_query, generate_response]

Task: "What was the best selling product last week?"

Plan: [fetch_top_products, generate_query, generate_response]

Task: {USER INPUT}

Plan:

Ci sono due cose da notare su questo esempio:

- Il formato del *plan* usato qui—una lista di *function* i cui *parameter* sono dedotti dall'*agent*—è solo uno dei tanti modi per strutturare il *control flow* dell'*agent*.
- La *function* `generate_query` prende la *history* corrente del *task* e gli *output* più recenti degli *tool* per generare una *query* da alimentare al *response generator*. L'*output* dello *tool* ad ogni passo viene aggiunto alla *history* del *task*.

Dato l'*user input* "Qual è il prezzo del prodotto più venduto la scorsa settimana", un *plan* generato potrebbe assomigliare a questo:

1. `get_time()`
2. `fetch_top_products()`
3. `fetch_product_info()`
4. `generate_query()`
5. `generate_response()`

Potresti chiederti: "E i *parameter* necessari per ogni *function*?" I *parameter* esatti sono difficili da prevedere in anticipo poiché sono spesso estratti dagli *output* degli *tool* precedenti. Se il primo passo, `get_time()`, restituisce "2030-09-13", allora l'*agent* può ragionare che i *parameter* per il passo successivo dovrebbero essere chiamati con i seguenti *parameter*:

```
retrieve_top_products(  
    start_date="2030-09-07",  
    end_date="2030-09-13",  
    num_products=1  
)
```

Spesso, non ci sono informazioni sufficienti per determinare i valori esatti dei *parameter* per una *function*. Ad esempio, se un utente chiede: "Qual è il prezzo medio dei prodotti più venduti?", le risposte alle seguenti domande non sono chiare:

- Quanti prodotti più venduti vuole considerare l'utente?
- L'utente vuole i prodotti più venduti della scorsa settimana, del mese scorso o di sempre?

Ciò significa che i modelli devono spesso indovinare, e le supposizioni possono essere sbagliate. Poiché sia la sequenza di azioni che i *parameter* associati sono generati da modelli di IA, possono essere *allucinati*. Le *allucinazioni* possono causare la chiamata di una *function* non valida o la chiamata di una *function* valida ma con *parameter* sbagliati. Le tecniche per migliorare le prestazioni di un modello in generale possono essere usate per migliorare le capacità di *planning* di un modello.

Ecco alcuni approcci per rendere un *agent* migliore nel *planning*:

- Scrivere un *system prompt* migliore con più esempi.
- Dare descrizioni migliori degli *tool* e dei loro *parameter* in modo che il modello li comprenda meglio.
- Riscrivere le *function* stesse per renderle più semplici, come rifattorizzare una *function* complessa in due *function* più semplici.
- Usare un modello più potente. In generale, i modelli più potenti sono migliori nel *planning*.
- Fare il *finetuning* di un modello per la *plan generation*.

Function calling. Molti fornitori di modelli offrono l'uso di *tool* per i loro modelli, trasformando efficacemente i loro modelli in *agent*. Un *tool* è una *function*. Invocare un *tool* è, quindi, spesso chiamato *function calling*. Diverse *API* di modelli funzionano in modo diverso, ma in generale, il *function calling* funziona come segue:

1. *Creare un tool inventory.*

Dichiarare tutti i *tool* che potresti voler far usare a un modello. Ogni *tool* è descritto dal suo *execution entry point* (ad es., il suo *function name*), i suoi *parameter* e la sua *documentation* (ad es., cosa fa la *function* e di quali *parameter* ha bisogno).

2. *Specificare quali tool l'agent può usare.*

Poiché *query* diverse potrebbero aver bisogno di *tool* diversi, molte *API* ti permettono di specificare un elenco di *tool* dichiarati da usare per *query*. Alcune ti permettono di controllare ulteriormente l'uso dei *tool* con le seguenti impostazioni:

required

Il modello deve usare almeno un *tool*.

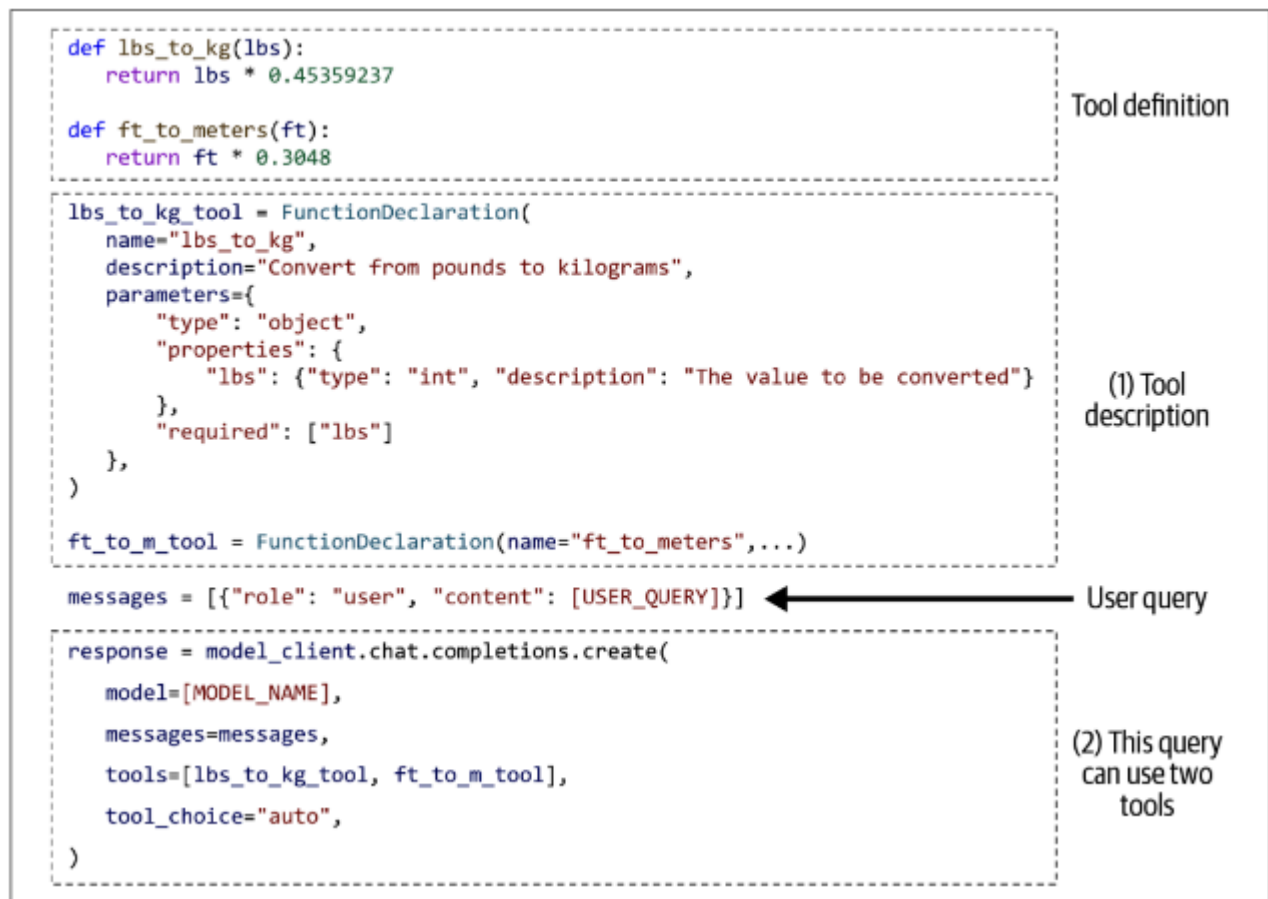
none

Il modello non dovrebbe usare alcun *tool*.

auto

Il modello decide quali *tool* usare.

Il *function calling* è illustrato nella Figura 6-10. Questo è scritto in *pseudocode* per renderlo rappresentativo di più *API*. Per usare una *API* specifica, si prega di fare riferimento alla sua documentazione.



[Figura 6-10. Un esempio di un modello che usa due tool semplici.]

Data una *query*, un *agent* definito come nella Figura 6-10 genererà automaticamente quali *tool* usare e i loro *parameter*. Alcune *API* di *function calling* si assicureranno che vengano generate solo *function* valide, anche se non saranno in grado di garantire i valori corretti dei *parameter*.

Ad esempio, data la *query* utente "Quanti chilogrammi sono 40 libbre?", l'*agent* potrebbe decidere che ha bisogno dello *tool* `lbs_to_kg_tool` con un valore di *parameter* di 40. La risposta dell'*agent* potrebbe assomigliare a questa:

```
response = ModelResponse(
    finish_reason='tool_calls',
    message=chat.Message(
        content=None,
        role='assistant',
        tool_calls=[
            ToolCall(
                function=Function(
                    arguments='{"lbs":40}',
                    name='lbs_to_kg'),
                type='function')
        ]
    )
)
```

Da questa risposta, puoi evocare la funzione `lbs_to_kg(lbs=40)` e usare il suo *output* per generare una risposta agli utenti.

Quando lavori con gli *agent*, chiedi sempre al sistema di segnalare quali valori dei *parameter* utilizza per ogni *function call*. Ispeziona questi valori per assicurarti che siano corretti.

Planning granularity. Un *plan* è una *roadmap* che delinea i passi necessari per realizzare un *task*. Una *roadmap* può avere diversi livelli di *granularity*. Per pianificare un anno, un *plan* trimestrale è di *livello superiore* rispetto a un *plan* mese per mese, che a sua volta è di *livello superiore* rispetto a un *plan* settimana per settimana.

C'è un compromesso *planning/execution*. Un *plan* dettagliato è più difficile da generare ma più facile da eseguire. Un *plan* di *livello superiore* è più facile da generare ma più difficile da eseguire. Un approccio per aggirare questo compromesso è pianificare *gerarchicamente*. Primo, usa un *planner* per generare un *plan* di *livello superiore*, come un *plan* trimestrale. Poi, per ogni trimestre, usa lo stesso o un *planner* diverso per generare un *plan* mese per mese.

Finora, tutti gli esempi di *plan* generati usano i nomi esatti delle *function*, che è molto *granulare*. Un problema con questo approccio è che il *tool inventory* di un *agent* può cambiare nel tempo. Ad esempio, la *function* per ottenere la data corrente `get_time()` può essere rinominata in `get_current_time()`. Quando uno *tool* cambia, dovrai aggiornare il tuo *prompt* e tutti i tuoi esempi. Usare i nomi esatti delle *function* rende anche più difficile riutilizzare un *planner* in diversi casi d'uso con diverse *API* di *tool*.

Se in precedenza hai *finetunato* un modello per generare *plan* basati sul vecchio *tool inventory*, dovrai *finetunare* nuovamente il modello sul nuovo *tool inventory*.

Per evitare questo problema, i *plan* possono anche essere generati usando un linguaggio più naturale, che è di *livello superiore* rispetto ai nomi di *function* specifici del dominio. Ad esempio, data la *query* "Qual è il prezzo del prodotto più venduto la scorsa settimana", a un *agent* può essere istruito di *outputtare* un *plan* che assomigli a questo:

1. ottenere la data corrente
2. recuperare il prodotto più venduto la scorsa settimana
3. recuperare le informazioni sul prodotto
4. generare la query
5. generare la risposta

Usare un linguaggio più naturale aiuta il tuo *plan generator* a diventare robusto ai cambiamenti nelle *API* degli *tool*. Se il tuo modello è stato *addestrato* principalmente su linguaggio naturale, sarà probabilmente migliore nel comprendere e generare *plan* in linguaggio naturale e meno probabile che *allucini*.

Lo svantaggio di questo approccio è che hai bisogno di un *translator* per tradurre ogni azione in linguaggio naturale in comandi eseguibili.¹³ Tuttavia, tradurre è un *task* molto più semplice che pianificare e può essere fatto da modelli più deboli con un minor rischio di *allucinazione*.

Complex plans. Gli esempi di *plan* finora sono stati *sequenziali*: l'azione successiva nel *plan* viene sempre eseguita dopo che l'azione precedente è terminata. L'ordine in cui le azioni possono essere eseguite è chiamato *control flow*. La forma *sequenziale* è solo un tipo di *control flow*. Altri tipi di *control flow* includono il *parallelo*, l'*if statement* e il *for loop*. La seguente lista fornisce una panoramica di ciascun *control flow*, incluso quello *sequenziale* per confronto:

Sequential

Eseguire il *task B* dopo che il *task A* è completo, probabilmente perché il *task B* dipende dal *task A*. Ad esempio, la *query SQL* può essere eseguita solo dopo che è stata tradotta dall'*input* in linguaggio naturale.

Parallel

Eseguire i *task A* e *B* contemporaneamente. Ad esempio, data la *query* "Trovami i prodotti più venduti sotto i \$100", un *agent* potrebbe prima recuperare i primi 100 prodotti più venduti e, per ognuno di questi prodotti, recuperare il suo prezzo.

If statement

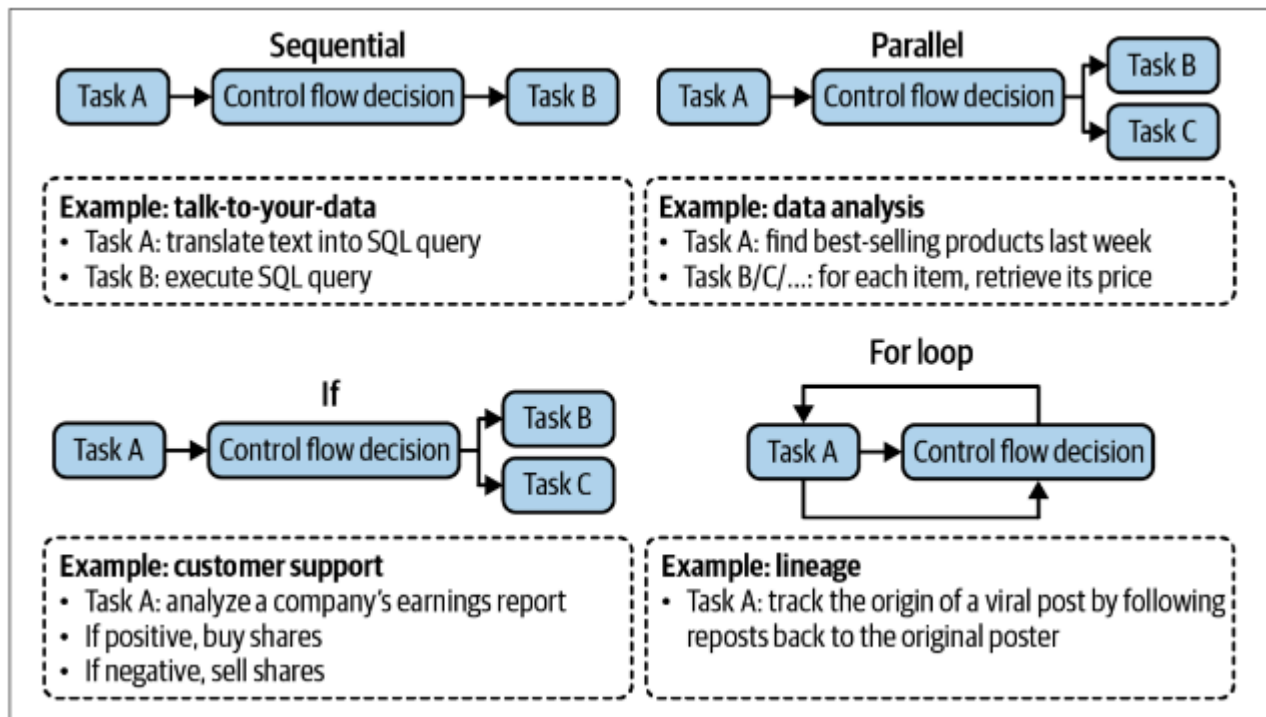
Eseguire il *task B* o il *task C* a seconda dell'*output* dal passo precedente. Ad esempio, l'*agent* controlla prima il rapporto sugli utili di NVIDIA. In base a questo rapporto, può poi decidere di vendere o comprare azioni NVIDIA.

For loop

Ripetere l'esecuzione del *task A* fino a quando una condizione specifica non è soddisfatta. Ad esempio, continuare a generare numeri casuali fino a quando non si ottiene un numero primo.

Questi diversi *control flow* sono visualizzati nella Figura 6-11.

¹³ *Chameleon* (Lu et al., 2023) chiama questo *translator* un *program generator*.



[Figura 6-11. Esempi di diversi ordini in cui un *plan* può essere eseguito.]

In *traditional software engineering*, le condizioni per i *control flow* sono esatte. Con gli *agent* alimentati da *IA*, i modelli di *IA* determinano i *control flow*. I *plan* con *control flow* non *sequenziali* sono più difficili sia da generare che da tradurre in comandi eseguibili.

Quando valuti un *framework* per *agent*, controlla quali *control flow* supporta. Ad esempio, se il sistema ha bisogno di navigare dieci siti web, può farlo simultaneamente? L'*execution parallela* può ridurre significativamente la *latency* percepita dagli utenti.

Reflection and error correction

Anche i *plan* migliori devono essere costantemente valutati e aggiustati per massimizzare le loro possibilità di successo. Sebbene la *reflection* non sia strettamente necessaria affinché un *agent* operi, è necessaria affinché un *agent* abbia successo.

La *reflection* può essere utile in molti punti durante un processo di *task*:

- Dopo aver ricevuto una *query* utente per valutare se la richiesta è fattibile.
- Dopo la *initial plan generation* per valutare se il *plan* ha senso.
- Dopo ogni *execution step* per valutare se è sulla strada giusta.
- Dopo che l'intero *plan* è stato eseguito per determinare se il *task* è stato realizzato.

La *reflection* e la *error correction* sono due meccanismi diversi che vanno di pari passo. La *reflection* genera *insights* che aiutano a scoprire errori da correggere.

La *reflection* può essere fatta con lo stesso *agent* usando *self-critique prompts*. Può anche essere fatta con un componente separato, come uno *scorer* specializzato: un modello che restituisce un punteggio concreto per ogni esito.

Proposta per la prima volta da *ReAct* (Yao et al., 2022), l'*interleaving reasoning and action* è diventato un modello comune per gli *agent*. Yao et al. hanno usato il termine "*reasoning*" per comprendere sia il *planning* che la *reflection*. Ad ogni passo, all'*agent* viene chiesto di spiegare il suo pensiero (*planning*), intraprendere azioni, poi analizzare le osservazioni (*reflection*), fino a quando il *task* non è considerato finito dall'*agent*. All'*agent* viene tipicamente *promptato*, usando esempi, per generare *output* nel seguente formato:

Thought 1: [Pensiero/Pianificazione del passo 1]

Act 1: [Azione del passo 1]

Observation 1: [Osservazione/Risultato del passo 1]

... [continua finché la *reflection* determina che il *task* è finito] ...

Thought N: [Pensiero finale]

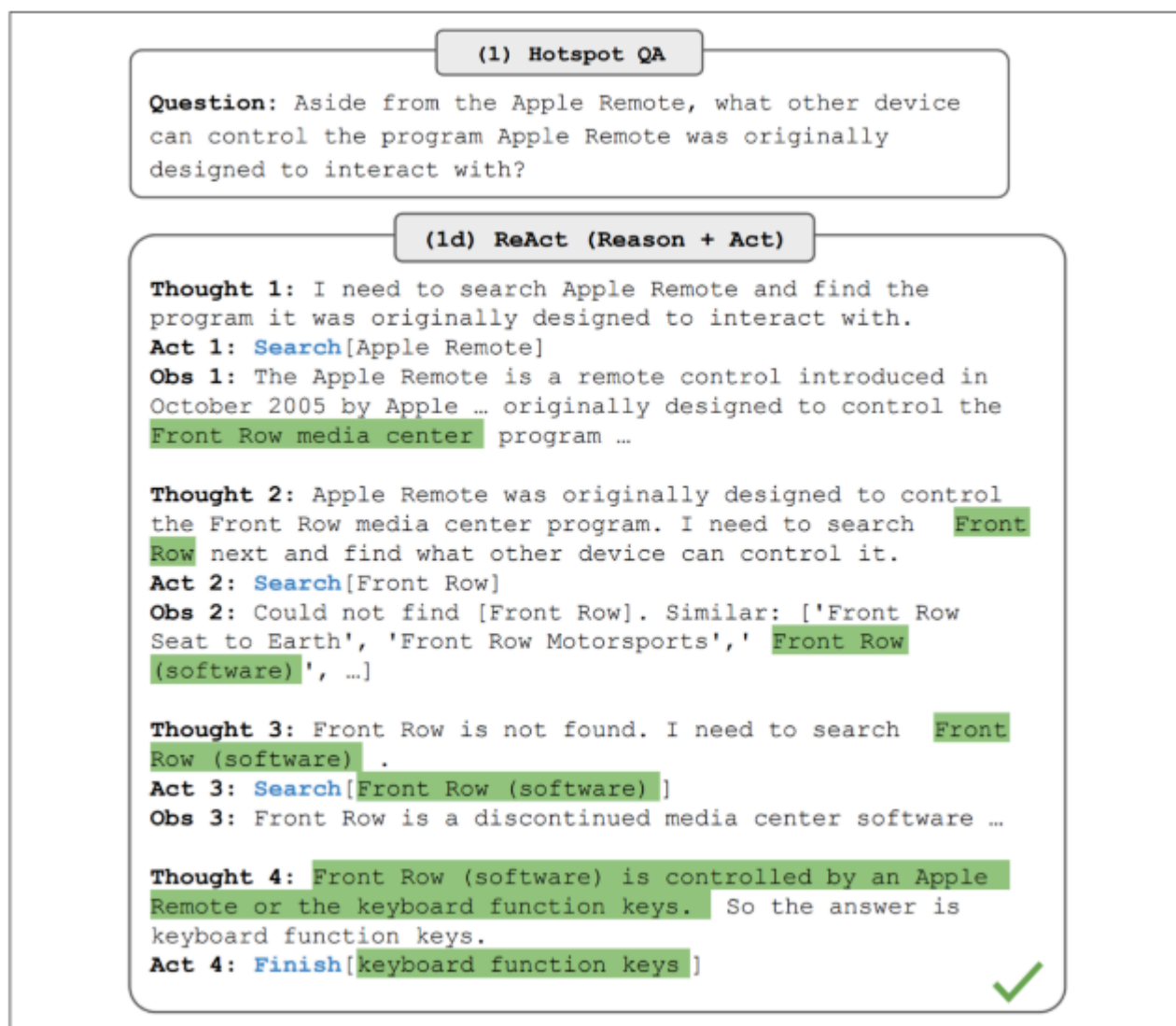
Act N: Finish [Risposta alla query]

La Figura 6-12 mostra un esempio di un *agent* che segue il *framework ReAct* che risponde a una domanda da *HotpotQA* (Yang et al., 2018), un *benchmark* per il *multi-hop question answering*.

Puoi implementare la *reflection* in un'impostazione *multi-agent*: un *agent* pianifica e intraprende azioni, e un altro *agent* valuta l'esito dopo ogni passo o dopo un certo numero di passi.¹⁴

Se la risposta dell'*agent* non è riuscita a realizzare il *task*, puoi *promptare* l'*agent* a riflettere sul perché ha fallito e su come migliorare. Basandosi su questo suggerimento, l'*agent* genera un nuovo *plan*. Questo permette agli *agent* di imparare dai loro errori. Ad esempio, dato un *task* di *coding generation*, un *evaluator* potrebbe valutare che il codice generato fallisce l'X% dei *test case*. L'*agent* poi riflette che la ragione del fallimento è perché non ha tenuto conto di array dove tutti i numeri sono negativi. L'*actor* poi genera un nuovo codice, tenendo conto di array tutti negativi.

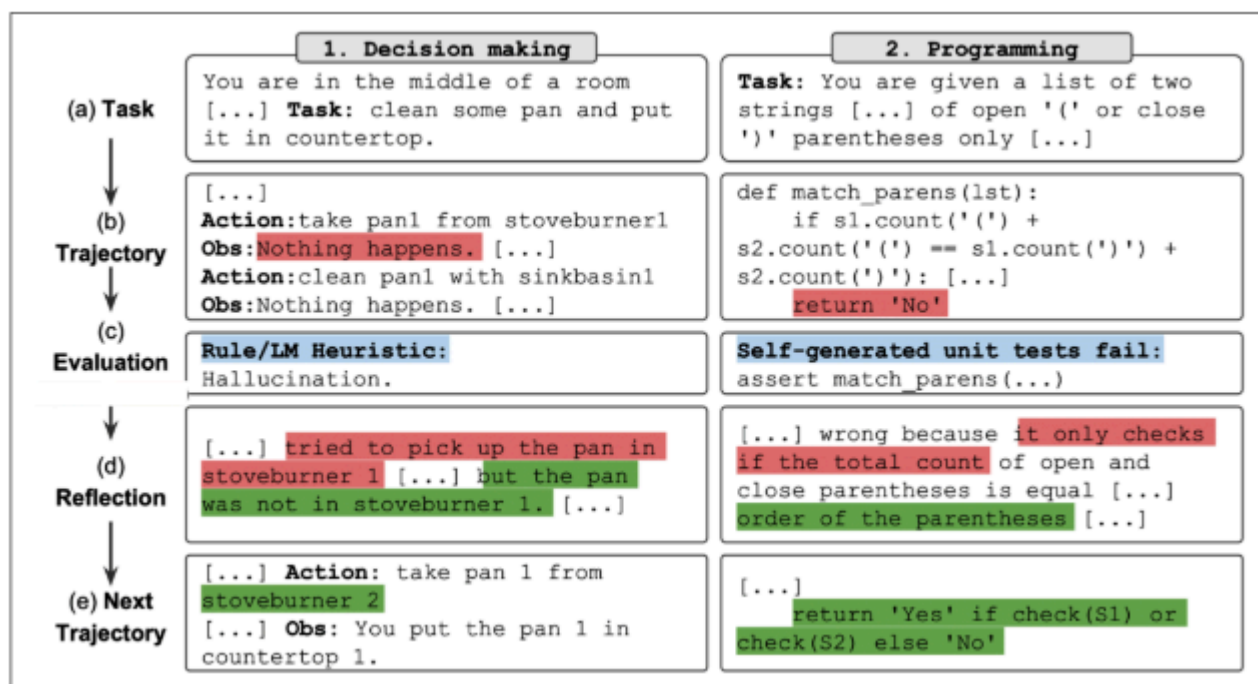
¹⁴ Questo mi ricorda il metodo *actor-critic (AC) agent* (Konda and Tsitsiklis, 1999) nel *reinforcement learning*.



[Figura 6-12. Un agent ReAct in azione. Immagine tratta dal documento ReAct (Yao et al., 2022). L'immagine è concessa in licenza CC BY 4.0.]

Questo è l'approccio che ha adottato *Reflexion* (Shinn et al., 2023). In questo *framework*, la *reflection* è separata in due moduli: un *evaluator* che valuta l'esito e un *self-reflection module* che analizza cosa è andato storto. La Figura 6-13 mostra esempi di *agent Reflexion* in azione. Gli autori hanno usato il termine "*trajectory*" per riferirsi a un *plan*. Ad ogni passo, dopo la *evaluation* e la *self-reflection*, l'*agent* propone una nuova *trajectory*.

Rispetto alla *plan generation*, la *reflection* è relativamente facile da implementare e può portare a un sorprendente miglioramento delle prestazioni. Lo svantaggio di questo approccio è la *latency* e il costo. I *Thoughts*, le *Observations* e talvolta le *Actions* possono richiedere molti *token* per essere generati, il che aumenta il costo e la *latency* percepita dall'utente, specialmente per *task* con molti passi intermedi. Per spingere i loro *agent* a seguire il formato, sia gli autori di *ReAct* che di *Reflexion* hanno usato molti esempi nei loro *prompt*. Questo aumenta il costo del calcolo degli *input token* e riduce lo spazio di *context* disponibile per altre informazioni.



[Figura 6-13. Esempi di come funzionano gli *agent Reflexion*. Immagini dal repository GitHub di *Reflexion*.]

Tool selection

Poiché gli *tool* spesso svolgono un ruolo cruciale nel successo di un *task*, la *tool selection* richiede un'attenta considerazione. I *tool* da dare al tuo *agent* dipendono dall'*environment* e dal *task*, ma dipendono anche dal modello di *IA* che alimenta l'*agent*.

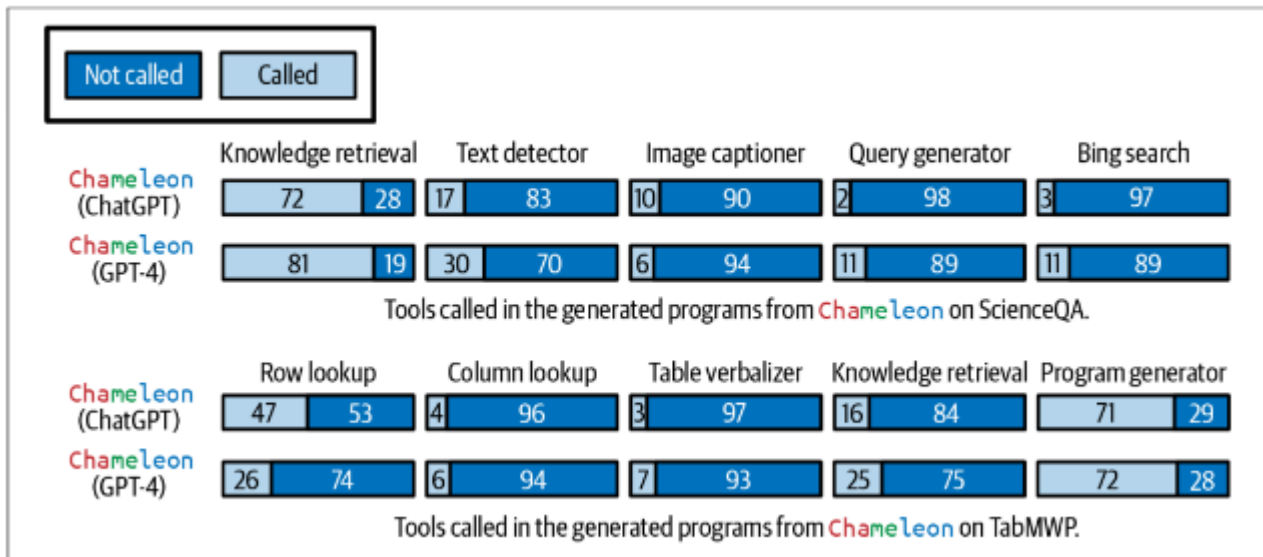
Non esiste una guida infallibile su come selezionare il miglior insieme di *tool*. La letteratura sugli *agent* consiste in un'ampia gamma di *tool inventory*. Ad esempio, *Toolformer* (Schick et al., 2023) ha *finetunato* *GPT-J* per imparare cinque *tool*. *Chameleon* (Lu et al., 2023) usa 13 *tool*. D'altra parte, *Gorilla* (Patil et al., 2023) ha tentato di *promptare* gli *agent* a selezionare la giusta *API call* tra 1.645 *API*.

Più *tool* danno all'*agent* più capacità. Tuttavia, più *tool* ci sono, più è difficile usarli in modo efficiente. È simile a come è più difficile per gli umani padroneggiare un grande insieme di *tool*. Aggiungere *tool* significa anche aumentare le descrizioni degli *tool*, che potrebbero non entrare nel *context* di un modello.

Come molte altre decisioni durante la costruzione di applicazioni di *IA*, la *tool selection* richiede sperimentazione e analisi. Ecco alcune cose che puoi fare per aiutarti a decidere:

- Confronta le prestazioni di un *agent* con diversi insiemi di *tool*.
- Fai uno *ablation study* per vedere quanto le prestazioni dell'*agent* calano se un *tool* viene rimosso dal suo *inventory*. Se un *tool* può essere rimosso senza un calo delle prestazioni, rimuovilo.

- Cerca *tool* su cui l'*agent* commette frequentemente errori. Se un *tool* si dimostra troppo difficile da usare per l'*agent*—ad esempio, un *prompting* estensivo e persino il *finetuning* non riescono a far sì che il modello impari a usarlo—cambia lo *tool*.
- Traccia la distribuzione delle *tool call* per vedere quali *tool* sono più utilizzati e quali sono meno utilizzati. La Figura 6-14 mostra le differenze nei modelli di utilizzo degli *tool* di *GPT-4* e *ChatGPT* in *Chameleon* (Lu et al., 2023).



[Figura 6-14. Diversi modelli e *task* esprimono diversi modelli di utilizzo degli *tool*. Immagine da Lu et al. (2023). Adattata da un'immagine originale concessa in licenza CC BY 4.0.]

Gli esperimenti di Lu et al. (2023) dimostrano anche due punti:

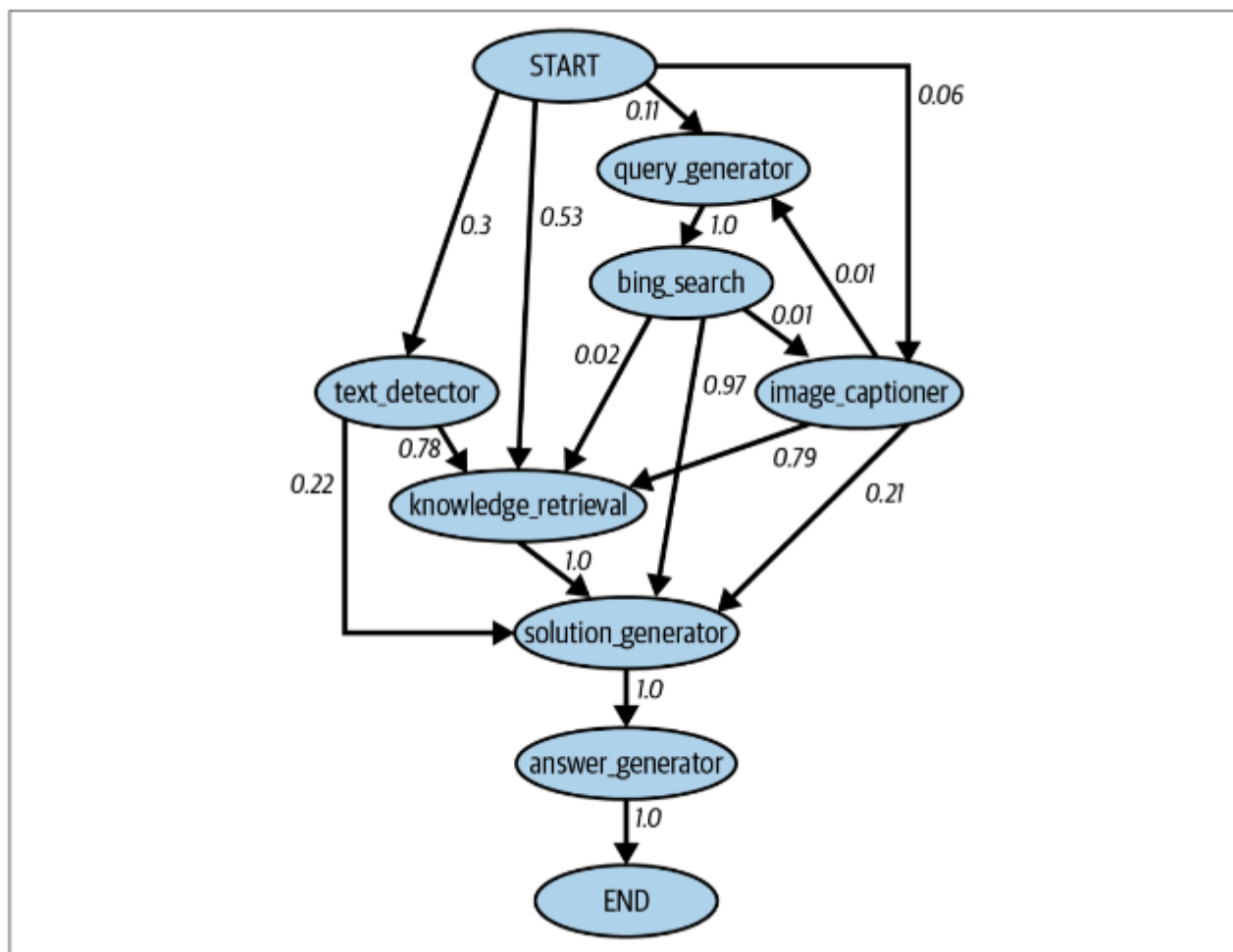
1. *Task* diversi richiedono *tool* diversi. *ScienceQA*, il *task* di risposta a domande scientifiche, si affida molto di più agli *tool* di *knowledge retrieval* rispetto a *TabMWP*, un *task* di risoluzione di problemi matematici tabellari.
2. Modelli diversi hanno preferenze diverse per gli *tool*. Ad esempio, *GPT-4* sembra selezionare un insieme più ampio di *tool* rispetto a *ChatGPT*. *ChatGPT* sembra favorire l'*image captioning*, mentre *GPT-4* sembra favorire la *knowledge retrieval*.

Quando valuti un *framework* per *agent*, valuta quali *planner* e *tool* supporta. *Framework* diversi potrebbero concentrarsi su diverse categorie di *tool*. Ad esempio, *AutoGPT* si concentra su *social media API* (Reddit, X e Wikipedia), mentre *Composio* si concentra su *enterprise API* (Google Apps, GitHub e Slack).

Poiché le tue esigenze probabilmente cambieranno nel tempo, valuta quanto è facile estendere il tuo *agent* per incorporare nuovi *tool*.

Come umani, diventiamo più produttivi non solo usando gli *tool* che ci vengono dati, ma anche creando *tool* progressivamente più potenti da quelli più semplici. L'IA può creare nuovi *tool* dai suoi *tool* iniziali?

Chameleon (Lu et al., 2023) propone lo studio della *tool transition*: dopo lo *tool X*, quanto è probabile che l'*agent* chiami lo *tool Y*? La Figura 6-15 mostra un esempio di *tool transition*. Se due *tool* sono frequentemente usati insieme, possono essere combinati in uno *tool* più grande. Se un *agent* è a conoscenza di questa informazione, l'*agent* stesso può combinare *tool* iniziali per costruire continuamente *tool* più complessi.



[Figura 6-15. Un albero di *tool transition* di Lu et al. (2023). Adattato da un'immagine originale concessa in licenza CC BY 4.0.]

Voyager (Wang et al., 2023) propone un *skill manager* per tenere traccia delle nuove *skill* (*tool*) che un *agent* acquisisce per un riutilizzo successivo. Ogni *skill* è un programma di codifica. Quando lo *skill manager* determina che una *skill* appena creata è utile (ad es., perché ha aiutato con successo un *agent* a realizzare un *task*), aggiunge questa *skill* alla *skill library* (concettualmente simile al *tool inventory*). Questa *skill* può essere recuperata in seguito per essere usata per altri *task*.

All'inizio di questa sezione, abbiamo menzionato che il successo di un *agent* in un *environment* dipende dal suo *tool inventory* e dalle sue capacità di *planning*. Fallimenti in entrambi gli aspetti possono causare il fallimento dell'*agent*. La prossima sezione discuterà le diverse *modalità di fallimento* di un *agent* e come valutarle per individuarle.

Agent Failure Modes and Evaluation

La *evaluation* riguarda il rilevamento dei fallimenti. Più complesso è il *task* che un *agent* esegue, più sono i possibili punti di fallimento. Oltre alle *modalità di fallimento* comuni a tutte le applicazioni di IA discusse nei Capitoli 3 e 4, gli *agent* hanno anche fallimenti unici causati dal *planning*, dall'*execution* degli *tool* e dall'*efficiency*. Alcuni dei fallimenti sono più facili da catturare di altri.

Per valutare un *agent*, identifica le sue *modalità di fallimento* e misura quanto spesso ciascuna di queste *modalità di fallimento* si verifica.

Ho creato un *benchmark* semplice per illustrare queste diverse *modalità di fallimento* che puoi vedere sul *repository GitHub* del libro. Ci sono anche *benchmark* e *leaderboard* per *agent* come il *Berkeley Function Calling Leaderboard*, l'*evaluation harness* di *AgentOps* e il *benchmark TravelPlanner*.

Planning failures

Il *planning* è difficile e può fallire in molti modi. La *modalità di fallimento* più comune del *planning* è il *tool use failure*. L'*agent* potrebbe generare un *plan* con uno o più di questi errori:

Invalid tool

Ad esempio, genera un *plan* che contiene *bing_search*, ma *bing_search* non è nel *tool inventory* dell'*agent*.

Valid tool, invalid parameters.

Ad esempio, chiama *lbs_to_kg* con due *parameter*. *lbs_to_kg* è nel *tool inventory* ma richiede solo un *parameter*, *lbs*.

Valid tool, incorrect parameter values

Ad esempio, chiama *lbs_to_kg* con un *parameter*, *lbs*, ma usa il valore 100 per *lbs* quando dovrebbe essere 120.

Un'altra *modalità di fallimento* del *planning* è il *goal failure*: l'*agent* non riesce a raggiungere l'obiettivo. Questo può essere perché il *plan* non risolve un *task*, o risolve il *task* senza seguire i *constraint*. Per illustrare questo, immagina di chiedere al modello di pianificare un viaggio di due settimane da San Francisco a Hanoi con un *budget* di \$5.000. L'*agent* potrebbe pianificare un viaggio da San Francisco a Ho Chi Minh City, o pianificare un viaggio di due settimane da San Francisco a Hanoi che supererà di molto il *budget*.

Un *constraint* comune che viene spesso trascurato dalla valutazione degli *agent* è il tempo. In molti casi, il tempo che un *agent* impiega conta meno, perché puoi assegnare un *task* a un *agent* e devi controllare solo quando ha finito. Tuttavia, in molti casi, l'*agent* diventa meno utile con il passare del tempo. Ad esempio, se chiedi a un *agent* di preparare una proposta di *grant* e l'*agent* la finisce dopo la scadenza del *grant*, l'*agent* non è molto utile.

Un'interessante *modalità di fallimento* del *planning* è causata da errori nella *reflection*. L'*agent* è convinto di aver realizzato un *task* quando non l'ha fatto. Ad esempio, chiedi all'*agent* di assegnare 50 persone a 30 camere d'albergo. L'*agent* potrebbe assegnare solo 40 persone e insistere che il *task* è stato realizzato.

Per valutare un *agent* per i *planning failures*, un'opzione è creare un *planning dataset* dove ogni esempio è una tupla (*task*, *tool inventory*). Per ogni *task*, usa l'*agent* per generare un numero *K* di *plan*. Calcola le seguenti metriche:

1. Su tutti i *plan* generati, quanti sono validi?
2. Per un dato *task*, quanti *plan* deve generare l'*agent*, in media, per ottenere un *plan* valido?
3. Su tutte le *tool call*, quante sono valide?
4. Quante volte vengono chiamati *tool* non validi?
5. Quante volte vengono chiamati *tool* validi con *parameter* non validi?
6. Quante volte vengono chiamati *tool* validi con valori di *parameter* incorretti?

Analizza gli *output* dell'*agent* per individuare schemi. Su quali tipi di *task* l'*agent* fallisce di più? Hai un'ipotesi sul perché? Con quali *tool* il modello commette frequentemente errori? Alcuni *tool* potrebbero essere più difficili da usare per un *agent*. Puoi migliorare la capacità di un *agent* di usare uno *tool* impegnativo con un *prompting* migliore, più esempi o il *fine-tuning*. Se tutto fallisce, potresti considerare di sostituire questo *tool* con qualcosa di più facile da usare.

Tool failures

I *tool failures* si verificano quando viene usato lo *tool* corretto, ma l'*output* dello *tool* è sbagliato. Una *modalità di fallimento* è quando uno *tool* restituisce semplicemente *output* sbagliati. Ad esempio, un *image captioner* restituisce una descrizione errata, o un *SQL query generator* restituisce una *query SQL* errata.

Se l'*agent* genera solo *plan* di *alto livello* ed è coinvolto un *translation module* nella traduzione da ogni azione pianificata a comandi eseguibili, i fallimenti possono verificarsi a causa di *translation errors*.

I *tool failures* possono anche verificarsi perché l'*agent* non ha accesso agli *tool* giusti per il *task*. Un esempio ovvio è quando il *task* coinvolge il recupero dei prezzi azionari correnti da *internet* e l'*agent* non ha accesso a *internet*.

I *tool failures* sono dipendenti dallo *tool*. Ogni *tool* deve essere testato indipendentemente. Stampa sempre ogni *tool call* e il suo *output* in modo da poterli ispezionare e valutare. Se hai un *translator*, crea *benchmark* per valutarlo.

Rilevare i *missing tool failures* richiede una comprensione di quali *tool* dovrebbero essere usati. Se il tuo *agent* fallisce frequentemente su un dominio specifico, questo potrebbe essere dovuto alla mancanza di *tool* per questo dominio. Lavora con esperti umani del dominio e osserva quali *tool* userebbero.

Efficiency

Un *agent* potrebbe generare un *plan* valido utilizzando gli *tool* giusti per realizzare un *task*, ma potrebbe essere inefficiente. Ecco alcune cose che potresti voler tracciare per valutare l'*efficiency* di un *agent*:

- Quanti passi richiede l'*agent*, in media, per completare un *task*?
- Quanto costa l'*agent*, in media, per completare un *task*?
- Quanto tempo richiede tipicamente ogni azione? Ci sono azioni che sono particolarmente dispendiose in termini di tempo o costose?

Puoi confrontare queste metriche con la tua *baseline*, che può essere un altro *agent* o un operatore umano. Quando confronti *AI agent* con *human agent*, tieni presente che umani e *IA* hanno modalità di operare molto diverse, quindi ciò che è considerato efficiente per gli umani potrebbe essere inefficiente per l'*IA*, e viceversa. Ad esempio, visitare 100 pagine *web* potrebbe essere inefficiente per un *human agent* che può visitare solo una pagina alla volta, ma banale per un *AI agent* che può visitare tutte le pagine *web* contemporaneamente.

In questo capitolo, abbiamo discusso in dettaglio come funzionano i sistemi *RAG* e *agent*. Entrambi i modelli spesso hanno a che fare con informazioni che superano il limite di *context* di un modello. Un sistema di *memory* che integra il *context* del modello nella gestione delle informazioni può migliorare significativamente le sue capacità. Esploriamo ora come funziona un sistema di *memory*.

Memory

La *memory* si riferisce ai meccanismi che permettono a un modello di conservare e utilizzare le informazioni. Un sistema di *memory* è particolarmente utile per applicazioni *knowledge-rich* come *RAG* e applicazioni *multi-step* come gli *agent*. Un sistema *RAG* si affida alla *memory* per il suo *context* arricchito, che può crescere in più turni man mano che recupera più informazioni. Un sistema *agentic* ha bisogno della *memory* per memorizzare istruzioni, esempi, *context*, *tool inventory*, *plan*, *output* degli *tool*, *reflection* e altro. Mentre *RAG* e *agent* pongono maggiori

richieste sulla *memory*, è benefica per qualsiasi applicazione di IA che richieda la conservazione delle informazioni.

Un modello di IA ha tipicamente tre principali meccanismi di *memory*:

Internal knowledge

Il modello stesso è un meccanismo di *memory*, poiché conserva la conoscenza dai dati su cui è stato *addestrato*. Questa conoscenza è la sua *internal knowledge*. La *internal knowledge* di un modello non cambia a meno che il modello stesso non venga aggiornato. Il modello può accedere a questa conoscenza in tutte le *query*.

Short-term memory

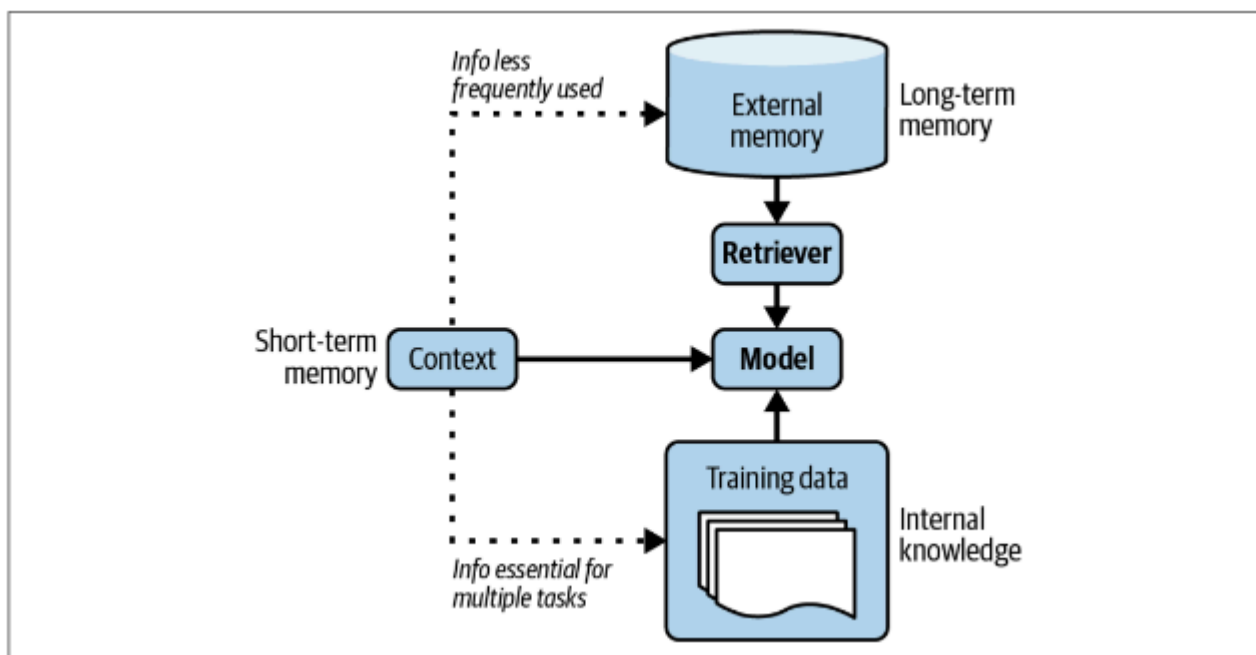
Il *context* di un modello è un meccanismo di *memory*. I messaggi precedenti in una conversazione possono essere aggiunti al *context* del modello, permettendo al modello di sfruttarli per generare risposte future. Il *context* di un modello può essere considerato la sua *short-term memory* in quanto non persiste tra *task* (*query*). È veloce da accedere, ma la sua capacità è limitata. Pertanto, è spesso usato per memorizzare informazioni che sono più importanti per il *task* corrente.

Long-term memory

Le fonti di dati esterne a cui un modello può accedere tramite *retrieval*, come in un sistema RAG, sono un meccanismo di *memory*. Questa può essere considerata la *long-term memory* del modello, in quanto può persistere tra *task*. A differenza della *internal knowledge* di un modello, le informazioni nella *long-term memory* possono essere cancellate senza aggiornare il modello.

Gli umani hanno accesso a meccanismi di *memory* simili. Come respirare è la tua *internal knowledge*. Tipicamente non dimentichi come respirare a meno che tu non sia in serio pericolo. La tua *short-term memory* contiene informazioni immediatamente rilevanti per ciò che stai facendo, come il nome di una persona che hai appena incontrato. La tua *long-term memory* è arricchita con libri, computer, appunti, ecc.

Quale meccanismo di *memory* usare per i tuoi dati dipende dalla sua frequenza d'uso. Le informazioni essenziali per tutti i *task* dovrebbero essere incorporate nella *internal knowledge* del modello tramite *training* o *finetuning*. Le informazioni che sono raramente necessarie dovrebbero risiedere nella sua *long-term memory*. La *short-term memory* è riservata a informazioni immediate, specifiche del *context*. Questi tre meccanismi di *memory* sono illustrati nella Figura 6-16.



[Figura 6-16. La gerarchia delle informazioni per un *agent*.]

La *memory* è essenziale per il funzionamento degli umani. Man mano che le applicazioni di *IA* si sono evolute, gli sviluppatori hanno rapidamente realizzato che la *memory* è importante anche per i modelli di *IA*. Sono stati sviluppati molti *tool* di *memory management* per modelli di *IA* e molti fornitori di modelli hanno incorporato *memory* esterne. Arricchire un modello di *IA* con un sistema di *memory* ha molti benefici. Ecco solo alcuni di essi:

Gestire l'overflow di informazioni all'interno di una sessione

Durante il processo di esecuzione di un *task*, un *agent* acquisisce molte nuove informazioni, che possono superare la *context length* massima dell'*agent*. Le informazioni in eccesso possono essere memorizzate in un sistema di *memory* con *long-term memory*.

Persistere le informazioni tra le sessioni

Un *AI coach* è praticamente inutile se ogni volta che vuoi un consiglio del *coach*, devi spiegare la storia della tua vita intera. Un *AI assistant* sarebbe fastidioso da usare se continuasse a dimenticare le tue preferenze. Avere accesso alla tua *conversation history* può permettere a un *agent* di personalizzare le sue azioni per te. Ad esempio, quando chiedi raccomandazioni di libri, se il modello ricorda che in passato hai amato *Il Problema dei Tre Corpi*, può suggerire libri simili.

Migliorare la coerenza di un modello

Se mi fai una domanda soggettiva due volte, come valutare una battuta tra 1 e 5, è molto più probabile che dia risposte coerenti se ricordo la mia risposta precedente. Allo stesso modo, se un modello di *IA* può fare riferimento alle sue risposte precedenti, può calibrare le sue risposte future per essere coerenti.

Mantenere l'integrità strutturale dei dati

Poiché il testo è intrinsecamente non strutturato, i dati memorizzati nel *context* di un modello basato su testo sono non strutturati. Puoi inserire dati strutturati nel *context*. Ad esempio, puoi inserire una tabella nel *context* riga per riga, ma non c'è garanzia che il modello capirà che questa dovrebbe essere una tabella. Avere un sistema di *memory* in grado di memorizzare dati strutturati può aiutare a mantenere l'*integrità strutturale* dei tuoi dati. Ad esempio, se chiedi a un *agent* di trovare potenziali *lead* di vendita, questo *agent* può sfruttare un foglio Excel per memorizzare i *lead*. Un *agent* può anche sfruttare una *queue* per memorizzare la sequenza di azioni da eseguire.

Un sistema di *memory* per modelli di IA consiste tipicamente in due funzioni:

- *Memory management*: gestire quali informazioni dovrebbero essere memorizzate nella *short-term* e *long-term memory*.
- *Memory retrieval*: recuperare informazioni rilevanti per il *task* dalla *long-term memory*.

Il *memory retrieval* è simile al *retrieval RAG*, poiché la *long-term memory* è una fonte di dati esterna. In questa sezione, mi concentrerò sul *memory management*. Il *memory management* consiste tipicamente in due operazioni: *add* e *delete memory*. Se l'archiviazione della *memory* è limitata, la *delete* potrebbe non essere necessaria. Questo potrebbe funzionare per la *long-term memory* perché l'archiviazione della *memory* esterna è relativamente economica e facilmente estensibile. Tuttavia, la *short-term memory* è limitata dalla *context length* massima del modello e, quindi, richiede una strategia per cosa aggiungere e cosa cancellare.

La *long-term memory* può essere usata per memorizzare l'*overflow* dalla *short-term memory*. Questa operazione dipende da quanto spazio vuoi allocare per la *short-term memory*. Per una data *query*, l'*input* del *context* nel modello consiste sia nella sua *short-term memory* che nelle informazioni recuperate dalla sua *long-term memory*. La capacità a *breve termine* di un modello è, quindi, determinata da quanto del *context* dovrebbe essere allocato per le informazioni recuperate dalla *long-term memory*. Ad esempio, se il 30% del *context* è riservato, allora il modello può usare al massimo il 70% del limite di *context* per la *short-term memory*. Quando questa soglia viene raggiunta, l'*overflow* può essere spostato nella *long-term memory*.

Come molti componenti discussi in precedenza in questo capitolo, il *memory management* non è unico per le applicazioni di IA. Il *memory management* è stato una pietra angolare di tutti i sistemi di dati e molte strategie sono state sviluppate per usare la *memory* in modo efficiente.

La strategia più semplice è *FIFO, first in, first out*. Il primo ad essere aggiunto alla *short-term memory* sarà il primo ad essere spostato nell'archiviazione esterna. Man mano che una conversazione si allunga, i fornitori di API come OpenAI potrebbero iniziare a rimuovere l'inizio della conversazione. Framework come *LangChain* potrebbero permettere la ritenzione degli ultimi N messaggi o degli ultimi N *token*. In una conversazione lunga, questa strategia assume che i primi

messaggi siano meno rilevanti per la discussione corrente. Tuttavia, questa ipotesi può essere fatalmente sbagliata. In alcune conversazioni, i messaggi più antichi potrebbero portare le informazioni più importanti, specialmente quando i primi messaggi dichiarano lo scopo della conversazione.¹⁵ Sebbene *FIFO* sia semplice da implementare, può causare la perdita di informazioni importanti da parte del modello.¹⁶

Strategie più sofisticate implicano la rimozione della ridondanza. I linguaggi umani contengono ridondanza per migliorare la chiarezza e compensare potenziali incomprensioni. Se c'è un modo per rilevare automaticamente la ridondanza, l'*impronta di memoria* sarà ridotta significativamente.

Un modo per rimuovere la ridondanza è usare un *riassunto* della conversazione. Questo riassunto può essere generato usando lo stesso o un altro modello. La *summarization*, insieme al *tracking* delle *named entities*, può fare molta strada. Bae et al. (2022) hanno fatto un ulteriore passo avanti. Dopo aver ottenuto il riassunto, gli autori volevano costruire una nuova *memory* unendo la *memory* con le informazioni chiave che il riassunto aveva perso. Gli autori hanno sviluppato un *classifier* che, per ogni frase nella *memory* e ogni frase nel riassunto, determina se solo una, entrambe o nessuna dovrebbe essere aggiunta alla nuova *memory*.

Liu et al. (2023), d'altra parte, hanno usato un approccio di *reflection*. Dopo ogni azione, all'*agent* viene chiesto di fare due cose:

1. Riflettere sulle informazioni che sono state appena generate.
2. Determinare se queste nuove informazioni dovrebbero essere inserite nella *memory*, dovrebbero fondersi con la *memory* esistente, o dovrebbero sostituire alcune altre informazioni, specialmente se le altre informazioni sono obsolete e contraddicono le nuove informazioni.

Quando si incontrano informazioni contraddittorie, alcune persone optano per mantenere quelle più recenti. Alcune persone chiedono ai modelli di *IA* di giudicare quale mantenere. Come gestire le contraddizioni dipende dal caso d'uso. Avere contraddizioni può causare confusione in un *agent* ma può anche aiutarlo ad attingere da diverse prospettive.

¹⁵ Per le conversazioni umane, potrebbe essere vero il contrario se i primi messaggi sono convenevoli.

¹⁶ Le strategie basate sull'uso, come rimuovere le informazioni meno frequentemente utilizzate, sono più impegnative, poiché avrai bisogno di un modo per sapere quando un modello utilizza un dato pezzo di informazione.

Riepilogo

Data la popolarità di *RAG* e il potenziale degli *agent*, i primi lettori hanno menzionato che questo è il capitolo che li entusiasma di più.

Questo capitolo è iniziato con *RAG*, il modello emerso per primo tra i due. Molti *task* richiedono una conoscenza di base estesa che spesso supera la *context window* di un modello. Ad esempio, i *code copilot* potrebbero aver bisogno di accedere a interi *codebase* e i *research assistant* potrebbero dover analizzare più libri. Sviluppato originariamente per superare le limitazioni di *context* di un modello, *RAG* permette anche un uso più efficiente delle informazioni, migliorando la qualità della risposta mentre riduce i costi. Dai primi giorni dei *foundation model*, era chiaro che il modello *RAG* sarebbe stato immensamente prezioso per un'ampia gamma di applicazioni, e da allora è stato rapidamente adottato in casi d'uso sia consumer che enterprise.

RAG impiega un processo in due fasi. Prima recupera le informazioni rilevanti dalla *memory* esterna e poi usa queste informazioni per generare risposte più accurate. Il successo di un sistema *RAG* dipende dalla qualità del suo *retriever*. I *retriever basati su termini*, come *Elasticsearch* e *BM25*, sono molto più leggeri da implementare e possono fornire *baseline* solide. I *retriever basati su embedding* sono più intensivi dal punto di vista computazionale ma hanno il potenziale per superare gli algoritmi *basati su termini*.

Il *retrieval basato su embedding* è alimentato dalla *vector search*, che è anche l'ossatura di molte applicazioni Internet fondamentali come la ricerca e i sistemi di raccomandazione. Molti algoritmi di *vector search* sviluppati per queste applicazioni possono essere usati per *RAG*.

Il modello *RAG* può essere visto come un caso speciale di *agent* in cui il *retriever* è uno *tool* che il modello può usare. Entrambi i modelli permettono a un modello di aggirare la sua limitazione di *context* e di rimanere più aggiornato, ma il modello *agentic* può fare anche di più. Un *agent* è definito dal suo *environment* e dagli *tool* a cui può accedere. In un *agent* alimentato da *IA*, l'*IA* è il *planner* che analizza il *task* assegnato, considera diverse soluzioni e sceglie quella più promettente. Un *task* complesso può richiedere molti passi per essere risolto, il che richiede un modello potente per pianificare. La capacità di un modello di pianificare può essere arricchita con la *reflection* e un sistema di *memory* per aiutarlo a tenere traccia dei suoi progressi.

Più *tool* dai a un modello, più capacità ha il modello, permettendogli di risolvere *task* più impegnativi. Tuttavia, più l'*agent* diventa automatizzato, più catastrofici possono essere i suoi fallimenti. L'uso di *tool* espone gli *agent* a molti rischi per la sicurezza discussi nel Capitolo 5. Affinché gli *agent* funzionino nel mondo reale, è necessario mettere in atto rigorosi meccanismi difensivi.

Sia *RAG* che gli *agent* lavorano con molte informazioni, che spesso superano la *context length* massima del modello sottostante. Ciò rende necessaria l'introduzione di un sistema di *memory* per gestire e utilizzare tutte le informazioni che un modello possiede. Questo capitolo si è concluso con una breve discussione su come appare questo componente.

Riepilogo | 305

RAG e gli *agent* sono entrambi metodi *basati su prompt*, poiché influenzano la qualità del modello esclusivamente attraverso gli *input* senza modificare il modello stesso. Sebbene possano abilitare molte applicazioni incredibili, modificare il modello sottostante può aprire possibilità ancora maggiori. Come farlo sarà l'argomento del prossimo capitolo.

